Software Requirements Specification

AI Client: OpenAI-Compatible AI Service Provisioning

Document ID: SRS-AI-CLIENT-001
Version: 0.17
Date: 2026-02-26
Standard: ISO/IEC/IEEE 29148:2018
Status: Draft
Author: Mark Sigler

Document Control

Revision History:

Version	Date	Author	Changes
0.1	2026-02-01	Mark Sigler	Initial document structure and executive summary
0.2	2026-02-02	Mark Sigler	Added personas, goals, and assumptions
0.3	2026-02-03	Mark Sigler	Added data flow architecture and security considerations
0.4	2026-02-04	Mark Sigler	Added functional requirements (initial set)
0.5	2026-02-05	Mark Sigler	Added configuration schema and error handling

Version	Date	Author	Changes
0.6	2026-02-06	Mark Sigler	Added UAT plan
0.7	2026-02-07	Mark Sigler	Added MCP support, BYOK via SecretStorage, Enterprise SSO graduation path
0.8	2026-02-08	Mark Sigler	Added prompt engineering patterns and structured output requirements
0.9	2026-02-09	Mark Sigler	Added function calling (tools) requirements and examples
0.10	2026-02-10	Mark Sigler	Added system architecture diagrams and component overview
0.11	2026-02-11	Mark Sigler	Added image generation requirements (limited scope: single image generation only)
0.12	2026-02-12	Mark Sigler	Added design-phase requirements (§7) — provider-agnostic interface, BYOK, MCP, enterprise graduation, architectural hedges
0.13	2026-02-13	Mark Sigler	P4 Priority tier: added 16 new requirements (REQ-68 through REQ-78, REQ-79-85) for URL

Version	Date	Author	Changes
			path construction, tool execution safety, user interaction patterns, concurrent requests, provider capability detection, multi-provider config, context window overflow handling
0.14	2026-02-14	Mark Sigler	P4 follow-up: added 3 extension lifecycle requirements (REQ-89, REQ-90, REQ-99)
0.15	2026-02-15	Mark Sigler	P4 NFR additions: added 9 new NFR rows and requirements (bundle size limit, CPU usage limit, i18n support, accessibility compliance, telemetry event schema, telemetry settings UI, SecretStorage API unavailable degradation path, response size limit, network timeouts, activation event constraints) — added REQ-07a, REQ-91-99
0.16	2026-02-16	Mark Sigler	P5 Test Coverage: added 8 new UAT test groups (19-26) covering provider-

Version	Date	Author	Changes
			specific compatibility, security edge cases, tool safety, performance/NFR validation, accessibility, telemetry, i18n, architectural hedge validation (47 new test scenarios)
0.17	2026-02-26	Mark Sigler	**IEEE 29148 Migration**: Restructured document into ISO/IEC/IEEE 29148:2018 format with modular file organization. All 99 requirements remapped to domain-prefixed IDs. Added formal traceability matrix. No content removed.

Table of Contents

Core Sections

1. **Introduction**
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Product Perspective
    - 1.4 Definitions, Acronyms, and Abbreviations
    - 1.5 References
2. **Overall Description**
    - 2.1 Product Functions
    - 2.2 User Classes and Characteristics

    - 2.3 Operating Environment
    - 2.4 Design and Implementation Constraints
    - 2.5 Assumptions and Dependencies
3. **System Features** *(Functional Requirements)*
    - 3.1 Configuration Management
    - 3.2 Credential Management (BYOK)
    - 3.3 API Integration
    - 3.4 Provider Management
    - 3.5 User Experience
    - 3.6 Prompt Construction & Context
    - 3.7 Structured Output
    - 3.8 Function Calling (Tools)
    - 3.9 Model Context Protocol (MCP)
    - 3.10 Enterprise Graduation
    - 3.11 Extension Lifecycle
4. **External Interface Requirements**
    - 4.1 User Interfaces
    - 4.2 Hardware Interfaces
    - 4.3 Software Interfaces
    - 4.4 Communications Interfaces
5. **Non-Functional Requirements**
    - 5.1 Performance Requirements
    - 5.2 Safety Requirements
    - 5.3 Security Requirements
    - 5.4 Software Quality Attributes
    - 5.5 Compliance Requirements
6. **Other Requirements**
    - 6.1 Database Requirements
    - 6.2 Internationalization Requirements
    - 6.3 Legal Requirements

Supporting Documentation

- **Appendix A: Requirements Traceability Matrix**
    - Migration cross-reference (REQ-XX → REQ-DOMAIN-XXX)
    - Forward traceability (Requirements → Tests)
    - Backward traceability (Tests → Requirements)
    - Coverage statistics
- **Appendix B: Glossary**
    - Complete definitions of all technical terms

- **Appendix C: Analysis Models**
  - System architecture diagrams
  - Data flow models
  - Sequence diagrams
- **Verification (UAT Plan)**
  - 26 test groups covering all functional and non-functional requirements
  - 130+ test scenarios
- **API Usage Patterns (Developer Guide)**
  - Prompt engineering best practices
  - Message role patterns
  - Structured output handling
  - Function calling examples
  - Context management strategies
- **Document Control**
  - Revision history
  - Approvals
  - License and attribution
  - Change control process

Document Approvals

Approvals:

Role	Name	Date
Product Owner	Mark Sigler	2026-02-26
Development Lead	[Pending]	[Pending]
QA Lead	[Pending]	[Pending]

License

This document is licensed under the **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)** license.

TASL Attribution: - **Title**: AI Client SRS - **Author**: Mark Sigler - **Source**: ai-client-srs.md - **License**: CC BY-SA 4.0

See LICENSE.md for full license text.

Generated in accordance with: ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering

Requirements Syntax: EARS (Easy Approach to Requirements Syntax)

Priority Framework: MoSCoW (Must have, Should have, Could have, Won't have)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements for the **AI Client Extension**: an OpenAI-compatible AI service provisioning system for IDE environments. This document is intended for:

- **Development Teams**: To understand what needs to be implemented
- **Quality Assurance**: To create test plans and verify requirements
- **Project Management**: To plan and track development
- **Stakeholders**: To validate

that needs are correctly captured - **Enterprise IT**: To evaluate security, deployment, and configuration requirements - **Solutions Engineers**: To support enterprise customer evaluations and deployments

Context

We are an enterprise software vendor providing development tools to enterprise customers. Our products—VS Code extensions, Eclipse plugins, and other IDE integrations—are installed on **developer workstations and laptops** (Windows, macOS, Linux). We are incorporating Generative AI features into these tools and need to provide flexibility for customers to configure any OpenAI API-compliant cloud service while maintaining enterprise-grade security and administrative control.

Strategic Rationale

A key driver for this architecture is **reducing friction in the evaluation phase of the sales cycle**. Prospective customers evaluating our AI-powered tools should not be required to stand up AI gateways, deploy proxy infrastructure, or configure

enterprise middleware before they can experience the product's value. The only prerequisites should be:

1. **An API key** (BYOK — Bring Your Own Key) for any OpenAI-compatible cloud service (OpenAI, Azure OpenAI, etc.)
2. **Network access** to that service from the developer's workstation

This "zero-infrastructure" evaluation path lets prospects go from installation to first AI interaction in minutes, not weeks—removing a common deal blocker in enterprise sales cycles where IT approval for infrastructure changes can stall evaluations indefinitely.

At the same time, the architecture provides a **clear graduation path to enterprise-grade AI governance**. When a customer moves from evaluation to production deployment, they can introduce an enterprise AI gateway, SSO, centralized policy controls, and observability—all through configuration changes alone, with **zero code modifications** to the extension. This dual-track design ensures we win evaluations quickly while meeting the security and compliance bar required for enterprise procurement.

1.2 Scope

Product Name

AI Client Extension — OpenAI-Compatible AI Service Provisioning for IDE Environments

Product Description

This project establishes a standardized client-side configuration framework for accessing OpenAI API-compliant cloud services from our IDE extensions. Configuration follows IDE best practices (JSON/YAML) and supports:

- **Provider Flexibility:** Configure any OpenAI API-compatible endpoint (Azure OpenAI, OpenAI, Anthropic Claude, enterprise gateways, or self-hosted Ollama)
- **Enterprise Control:** IT administrators can deploy managed configuration profiles
- **User Customization:** Individual developers can override settings where permitted

- **Cross-Platform:** Consistent configuration across Windows, macOS, and Linux

In Scope

The following capabilities are included in version 1.0:

Use Case	API Endpoint / Protocol	Status
Text Generation	`/v1/chat/completions`	✅ In Scope
Code Generation	`/v1/chat/completions`	✅ In Scope
Structured Output	`/v1/chat/completions` (JSON mode)	✅ In Scope
Function Calling	`/v1/chat/completions` (tools)	✅ In Scope
Image Generation	`/v1/images/generations`	✅ In Scope (limited: single image generation only)
Model Context Protocol (MCP)	MCP Server via `vscode.lm` API	✅ In Scope
BYOK / SecretStorage	VS Code `SecretStorage` API	✅ In Scope
Enterprise SSO	`vscode.authentication` provider	✅ In Scope

Note: Image generation is limited to single image generation only. Batch generation, image editing (`/v1/images/edits`), and image variations (`/v1/images/variations`) are not supported in v1.

Out of Scope (v1)

The following items are explicitly **not included** in the v1 release:

Item	Rationale	Future Version
	Complex stateful interactions requiring	Deferred indefinitely

Item	Rationale	Future Version
Agents / Assistants API	persistent threads, file storage, and code interpreter runtime; fundamentally different architecture from v1 stateless tool-call model	
LangGraph / Agent Frameworks	Local or remote agent runtimes require stateful orchestration, Python runtimes, and human-in-the-loop UX patterns beyond v1 scope	v2+
Audio/ Speech APIs	`/v1/audio/*` endpoints not needed for dev tools	N/A
Embeddings API	Search/RAG use cases deferred	v1.1
AWS Bedrock Native Auth (SigV4)	Requires AWS SDK dependency; customers can access Bedrock via enterprise gateway or OpenAI-compatible endpoint	v1.1
Central Configuration Management	Enterprise profile distribution from control point	v1.1
Admin Console / UI	Configuration via JSON/ YAML sufficient for v1	v1.1
Usage Analytics Dashboard	Customers use provider's own dashboards	v2
Prompt Templates / Library	Would require content storage	v2

Customer Responsibility

The following capabilities are the responsibility of the customer and their AI provider:

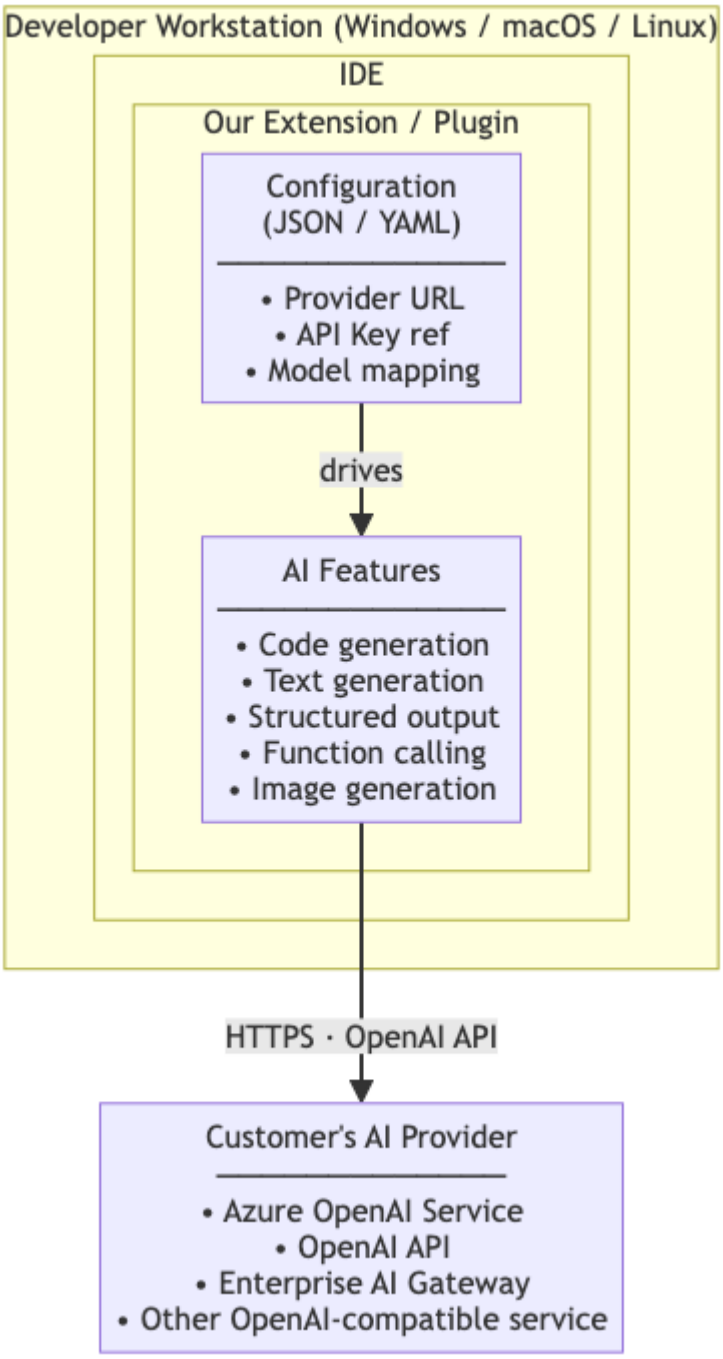
Item	Notes
Content Filtering	Configured at provider level (Azure Content Safety, OpenAI moderation)
Usage Monitoring	Use provider dashboards (Azure Portal, OpenAI Usage)
Rate Limiting	Provider-side limits; customer manages quotas with their provider

---

1.3 Product Perspective

System Context

The AI Client Extension operates as a client-side component installed on developer workstations, mediating between IDE features and customer-configured AI providers:



Key Benefits

- **Unlock Azure-first customers:** Large enterprises with existing Azure OpenAI deployments
- **Reduce support burden:** Consistent configuration patterns reduce “it doesn’t work” tickets
- **Pass security reviews:** No hardcoded credentials; enterprise credential patterns supported
- **Enable IT control:** Managed configuration profiles for enterprise-wide deployment

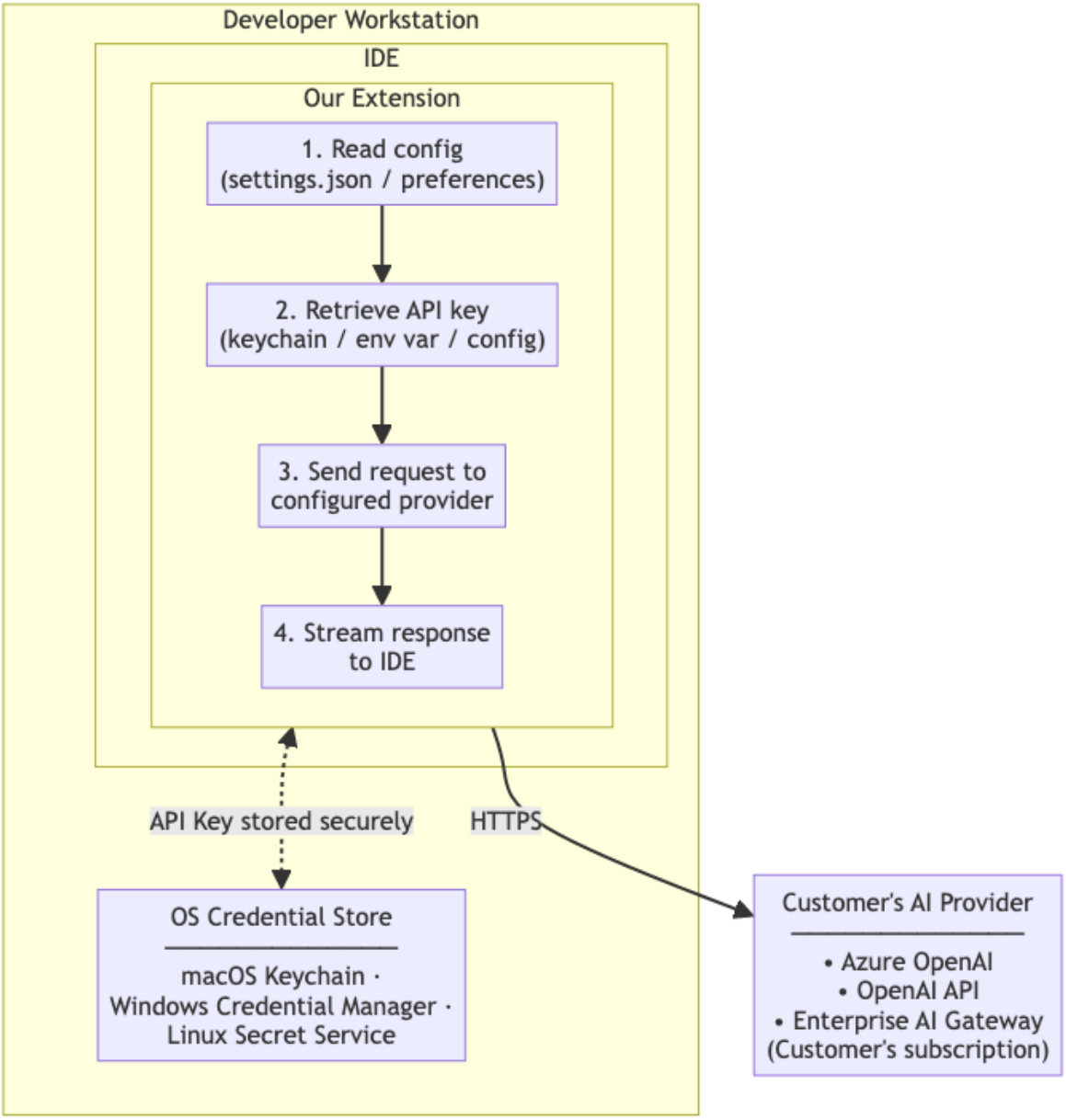
- **Single codebase:** No per-provider code paths; configuration-driven provider selection

Value Proposition by Stakeholder

Stakeholder	Value Delivered
Enterprise IT	Centrally manage AI provider configuration; enforce approved endpoints
Developers	AI features work out-of-the-box with their organization's approved provider
Security Teams	No hardcoded credentials; supports enterprise credential management
Our Product	Single codebase works with any OpenAI-compatible provider
Our Engineering	No per-provider code paths; configuration-driven provider selection

Data Flow Architecture

Our extension runs on the developer's workstation and communicates directly with the customer's configured AI provider:



What We (the Vendor) See

Data Type	Do We See It?	Notes
Prompts / Code	❌ No	Sent directly from workstation to customer's provider
Completions	❌ No	Returned directly from provider to workstation

Data Type	Do We See It?	Notes
API Keys	❌ No	Stored on customer's machine; never transmitted to us
Configuration	❌ No	Local to customer's machines
Usage Telemetry	⚠️ Only if opted-in	Optional, anonymized telemetry for product improvement

---

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
BYOK	Bring Your Own Key — user-supplied credentials
EARS	Easy Approach to Requirements Syntax
IDE	Integrated Development Environment (e.g., VS Code, Eclipse)
LLM	Large Language Model
MCP	Model Context Protocol — standard for tool/resource exposure to AI models
NFR	Non-Functional Requirement
OpenAI API	REST API specification for chat completions, originally by OpenAI, now widely adopted
Provider	An AI service endpoint (Azure OpenAI, OpenAI, enterprise gateway, Ollama, etc.)

Term	Definition
SecretStorage	VS Code API for secure credential storage using OS-level keychains
SSE	Server-Sent Events — HTTP streaming protocol for incremental responses
SSO	Single Sign-On
SRS	Software Requirements Specification
UAT	User Acceptance Testing
Universal LLM Client	HTTP client adhering to OpenAI API wire format, provider-agnostic

For complete glossary with 40+ terms, see [Appendix B: Glossary](#).

1.5 References

Normative References

- **ISO/IEC/IEEE 29148:2018** — Systems and software engineering — Life cycle processes — Requirements engineering
- **OpenAI API Reference** — https://platform.openai.com/docs/api-reference
- **OpenAI API Concepts** — https://platform.openai.com/docs/concepts
- **Model Context Protocol (MCP) Specification** — https://modelcontextprotocol.io/
- **VS Code Extension API** — https://code.visualstudio.com/api
  - `SecretStorage` API — https://code.visualstudio.com/api/references/vscode-api#SecretStorage
  - `Authentication` API — https://code.visualstudio.com/api/references/vscode-api#authentication
  - MCP Integration APIs

Informative References

- **Azure OpenAI Service Documentation** — https://learn.microsoft.com/en-us/azure/ai-services/openai/
- **vLLM Documentation** — https://docs.vllm.ai/
- **EARS (Easy Approach to Requirements Syntax)** — Mavin, A. et al. (2009)

- **MoSCoW Prioritization** — Agile requirements prioritization framework
- **RFC 2119** — Key words for use in RFCs to Indicate Requirement Levels

---

[Back to main document](#) | [Next: Overall Description →](#)

2. Overall Description

2.1 Product Functions

This project establishes a standardized client-side configuration framework for accessing OpenAI API-compliant cloud services from our IDE extensions. The extension runs on the developer's workstation and communicates directly with the customer's configured AI provider:

Core Functions:

- **Provider Flexibility:** Configure any OpenAI API-compatible endpoint (Azure OpenAI, OpenAI, Anthropic Claude, enterprise gateways, or self-hosted Ollama)
- **Enterprise Control:** IT administrators can deploy managed configuration profiles
- **User Customization:** Individual developers can override settings where permitted
- **Cross-Platform:** Consistent configuration across Windows, macOS, and Linux
- **Secure Credential Management:** API keys stored in OS-level secure storage via VS Code `SecretStorage` API
- **Tool Integration:** Model Context Protocol (MCP) support for agentic capabilities
- **Enterprise Graduation:** Zero-code-change transition from evaluation (BYOK) to production (enterprise gateway + SSO)

Supported Use Cases:

Use Case	API Endpoint / Protocol	Status
Text Generation	`/v1/chat/completions`	✅ In Scope
Code Generation	`/v1/chat/completions`	✅ In Scope

Use Case	API Endpoint / Protocol	Status
Structured Output	`/v1/chat/completions` (JSON mode)	✅ In Scope
Function Calling	`/v1/chat/completions` (tools)	✅ In Scope
Image Generation	`/v1/images/generations`	✅ In Scope (limited: single image generation only; batch generation, image editing, and variations not supported in v1)
Model Context Protocol (MCP)	MCP Server via `vscode.lm` API	✅ In Scope (v0.7)
BYOK / SecretStorage	VS Code `SecretStorage` API	✅ In Scope (v0.7)
Enterprise SSO	`vscode.authentication` provider	✅ In Scope (v0.7)
Agents / Assistants API	`/v1/assistants/*`	❌ Out of Scope
Embeddings	`/v1/embeddings`	⚠️ Future
Audio/Speech	`/v1/audio/*`	❌ Out of Scope

Value Proposition:

Stakeholder	Value Delivered
Enterprise IT	Centrally manage AI provider configuration; enforce approved endpoints
Developers	AI features work out-of-the-box with their organization's approved provider

Stakeholder	Value Delivered
Security Teams	No hardcoded credentials; supports enterprise credential management
Our Product	Single codebase works with any OpenAI-compatible provider
Our Engineering	No per-provider code paths; configuration-driven provider selection

---

2.2 User Classes and Characteristics

Customer-Side Personas

Persona	Role	Context	Primary Need
IT Administrator	Manages developer tooling	Rolling out our extension to 500+ developers	“Let me define one configuration and push it to all machines”
Security Engineer	Reviews developer tools	Evaluating our extension for approval	“Show me API keys are not stored in plaintext; show me the data flow”
Developer (Power User)	Uses AI features extensively	Wants to customize model settings	“Let me use a different model for certain tasks, if IT allows”
Developer (Typical)	Uses AI features casually	Just wants it to work	“I don’t want to configure anything; it should just work with our company’s AI”

Internal Personas

Persona	Role	Primary Need
Product Manager	Owns AI feature adoption	Unblock enterprise deals with flexible provider support
Extension Developer		Clean API to call AI services without provider-specific code

Persona	Role	Primary Need
	Builds VS Code/Eclipse plugins	
Solutions Engineer	Supports enterprise customers	Clear configuration docs; managed deployment options
Technical Writer	Creates user/admin docs	Configuration reference for all platforms and IDEs

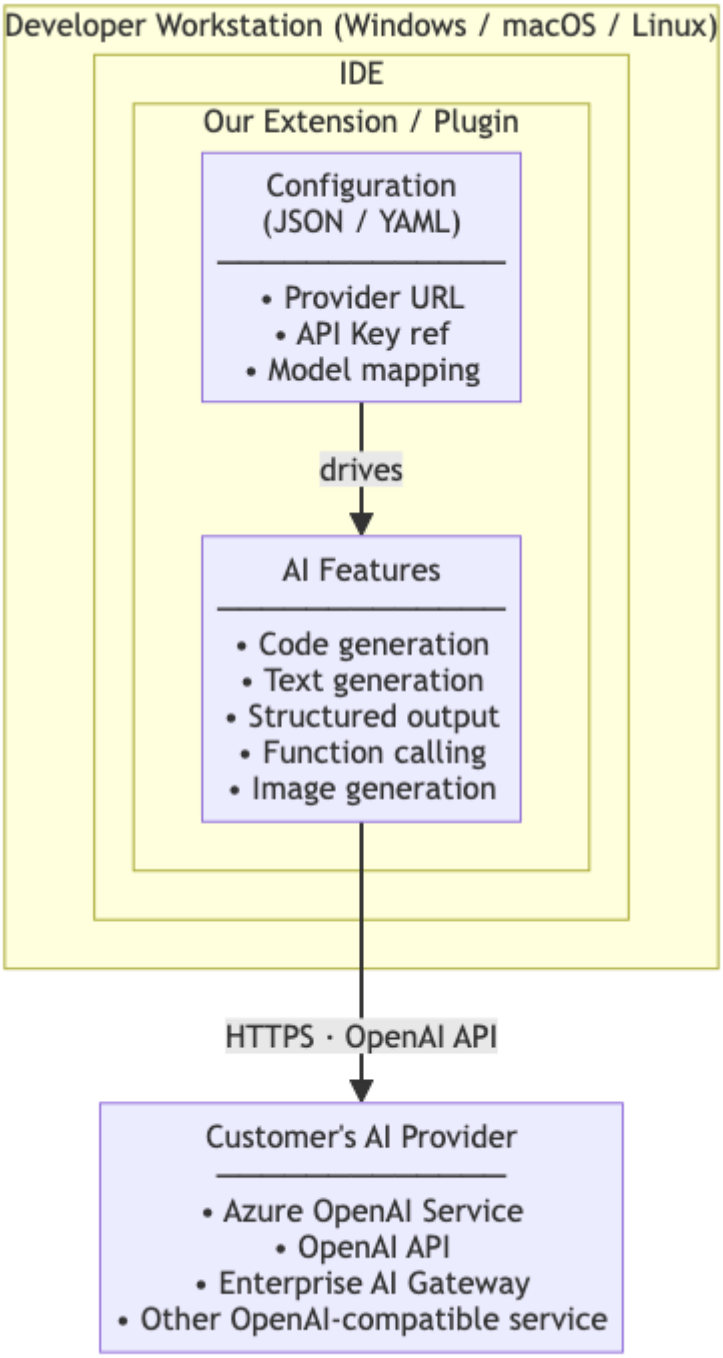
---

2.3 Operating Environment

Deployment Context

Our product is an **IDE extension** installed on developer workstations and laptops (Windows, macOS, Linux). We are incorporating Generative AI features into these tools and need to provide flexibility for customers to configure any OpenAI API-compliant cloud service while maintaining enterprise-grade security and administrative control.

Deployment Model



Strategic Rationale

A key driver for this architecture is **reducing friction in the evaluation phase of the sales cycle**. Prospective customers evaluating our AI-powered tools should not be required to stand up AI gateways, deploy proxy infrastructure, or configure

enterprise middleware before they can experience the product's value. The only prerequisites should be:

1. **An API key** (BYOK — Bring Your Own Key) for any OpenAI-compatible cloud service (OpenAI, Azure OpenAI, etc.)
2. **Network access** to that service from the developer's workstation

This "zero-infrastructure" evaluation path lets prospects go from installation to first AI interaction in minutes, not weeks—removing a common deal blocker in enterprise sales cycles where IT approval for infrastructure changes can stall evaluations indefinitely.

At the same time, the architecture provides a **clear graduation path to enterprise-grade AI governance**. When a customer moves from evaluation to production deployment, they can introduce an enterprise AI gateway, SSO, centralized policy controls, and observability—all through configuration changes alone, with **zero code modifications** to the extension (see §2.4 Enterprise Graduation requirements). This dual-track design ensures we win evaluations quickly while meeting the security and compliance bar required for enterprise procurement.

Supported Platforms

Platform	Versions	Notes
Windows	10+	x64 and ARM64
macOS	11+	Intel and Apple Silicon
Linux	Ubuntu 20.04+, RHEL 8+, Debian 11+	x64 and ARM64
VS Code	Latest 3 major versions	Primary IDE target
Eclipse	Latest 2 major versions	Secondary IDE target

2.4 Design and Implementation Constraints

Provider-Agnostic Interface (Wire Protocol)

Requirement: The extension must use a **Universal LLM Client** that communicates via the OpenAI Chat Completions wire format.

Implementation Constraints:

- **No vendor-specific SDKs** as the primary integration path. Do not depend on `@openai/openai` or `@anthropic-ai/sdk` directly. Use plain HTTP (Node.js `fetch` or equivalent) against the OpenAI-compatible REST surface.
- Target a configurable **`BASE_URL`** and **`MODEL_NAME`**.
- By default, BASE_URL should be editable in VS Code Settings (e.g., `https://api.openai.com/v1` for evaluation or `https://gateway.acme.com/v1` for enterprise).
- **URL Path Handling:** The BASE_URL must be the **complete base path** to which API-specific paths (e.g., `/chat/completions`, `/images/generations`) are appended. For Azure OpenAI, this includes the deployment: `https://<resource>.openai.azure.com/openai/deployments/<deployment-name>`. The extension appends `/chat/completions` (or other endpoint paths) to construct the full request URL.
- The extension must pass through custom headers and query parameters (e.g., Azure `api-version`) without hard-coding provider-specific logic.

Frictionless BYOK — Secure Key Storage

Requirement: Enable users to supply their own API keys without requiring backend infrastructure.

Implementation Constraints:

- Utilize the **VS Code `SecretStorage` API** (`context.secrets`) to persist API keys locally on the user's machine. **Never** store keys in `globalState`, `workspaceState`, or standard JSON settings files.
- Support **Header Injection**: The extension must allow custom authentication headers (e.g., `X-Api-Key`, `Authorization: Bearer …`) to be dynamically populated from `SecretStorage` at request time.
- Provide a first-run "Set API Key" command that writes to `SecretStorage` and confirms success.

Tooling & Agency (Model Context Protocol)

Requirement: Use the **Model Context Protocol (MCP)** as the standard for tool capabilities. **No local AI agent runtime is required.**

Dual MCP Role:

The extension serves two complementary MCP roles:

1. **MCP Server** — The extension registers as an MCP Server (via `vscode.lm.registerMcpServerDefinitionProvider`), exposing tools that *other* MCP hosts in the VS Code ecosystem (e.g., GitHub Copilot) can discover and invoke.
2. **Tool-Call Orchestrator** — For its *own* LLM interactions, the extension contains a lightweight orchestration loop that iteratively executes `tool_calls` returned by the cloud model via the Chat Completions API until the model produces a final text response.

Critically, the cloud-hosted model (e.g., GPT-4, Claude) acts as its own planner. The extension is a **stateless executor** — it does not run a local agent, reasoning framework, or planning engine. "Agent-like" multi-step behavior emerges from the model's native ability to emit `tool_calls` iteratively, with the extension mechanically executing and returning results.

Implementation Constraints:

- Register the extension as an **MCP Server** using the `vscode.lm.registerMcpServerDefinitionProvider` API.
- Expose extension-specific capabilities as **MCP Tools**, including (but not limited to):
  - `search_codebase` — workspace-wide text / symbol search
  - `run_unit_tests` — execute test suites and return results
  - `read_file_context` — read file contents with line ranges
- **MCP Tool Permission Model:** Tools exposed via MCP Server must be classified by risk level (`safe`, `read-only`, `write-only`, `destructive`). The extension must maintain an allowlist of external MCP hosts authorized to invoke each tool category. By default, only `safe` and `read-only` tools are externally visible. Destructive tools (e.g., `run_terminal_command`) are internal-only unless explicitly authorized.
- **Workspace Trust Integration:** In untrusted workspaces (per VS Code Workspace Trust API), the extension must disable tool execution, limit model interactions to read-only context (e.g., open file contents only), and suppress `run_unit_tests` and `run_terminal_command` tools entirely.
- **Reasoning vs. Execution model:** The cloud model is the brain (decides *what* to do). The extension is the hands (decides *how* to do it safely). The tool-call orchestration loop sends tool definitions to the model, executes any returned `tool_calls` locally via the VS Code Extension API, appends results as `tool` role messages, and re-calls the model — repeating until a final text response is produced or a maximum iteration limit is reached.

- **Tool Execution Path:** When the extension's own tool-call orchestrator invokes tools for its internal LLM interactions, it calls tool implementations **directly** (internal function calls), not via the MCP protocol. The MCP Server registration is solely for exposing tools to *external* MCP hosts (e.g., GitHub Copilot). This avoids protocol overhead and serialization round-trips for internal tool use.
- MCP tool definitions must include JSON Schema parameter descriptions to maximize model accuracy.
- The tool-call orchestration loop must be implemented as a **modular, replaceable component** (`ToolOrchestrator` interface) to support future substitution with more advanced orchestration backends (see Future-Proofing constraints below).

Path to Enterprise — The Gateway Graduation

Requirement: The architecture must support a **zero-code-change transition** from direct SaaS access to enterprise-governed access.

Implementation Constraints:

- **SSO Integration:** Implement support for the `vscode.authentication` provider API. When a BASE_URL points to an enterprise gateway, the extension should attempt to exchange a VS Code session token for a gateway bearer token via OIDC / OAuth flow.
- **Telemetry & Observability:** Include a `X-Request-Id` header (UUID v4) in **all** outgoing LLM calls to allow enterprise IT to trace requests from the IDE through their AI Gateway and into the upstream provider.
- **Proxy awareness:** Respect VS Code's built-in proxy settings and `HTTP_PROXY` / `HTTPS_PROXY` environment variables for corporate network environments.

Evaluation ⊡ Enterprise Specification Summary

Component	Evaluation Requirement (BYOK)	Enterprise Requirement (Production)
Auth	User-pasted key in `SecretStorage`	OIDC / OAuth via `vscode.authentication`
Routing	Direct to SaaS (OpenAI, Anthropic, Azure OpenAI)	Internal AI Gateway (Kong, Cloudflare, etc.)

Component	Evaluation Requirement (BYOK)	Enterprise Requirement (Production)
Protocol	OpenAI-Compatible REST + MCP for Tools	OpenAI-Compatible REST + MCP for Tools
Connectivity	Public Internet (Node.js `fetch`)	Corporate VPN / Proxy aware
Telemetry	Optional opt-in	`X-Request-Id` tracing through gateway
Key Storage	VS Code `SecretStorage`	`SecretStorage` + SSO token exchange

Future-Proofing: Architectural Hedges for Agent Frameworks

> **Context:** While local/remote agent frameworks (e.g., LangGraph, CrewAI) are explicitly **out of scope** for v1, the following low-cost architectural hedges ensure the extension can adopt advanced orchestration in the future with minimal refactoring. These are *design constraints*, not features — they impose structure on v1 implementation to preserve optionality.

1. Modular Tool Orchestrator

- The tool-call loop (send tools → receive `tool_calls` → execute → return results → repeat) must be encapsulated behind a **`ToolOrchestrator` interface**. The v1 implementation is a simple iterative loop driven by Chat Completions `tool_calls`. In a future version, this interface could be swapped for a LangGraph-backed orchestrator or cloud-hosted agent endpoint — with no changes to the rest of the extension.

2. Conversation Thread IDs

- Every conversation or multi-turn interaction must be assigned a **`thread_id`** (UUID v4), even though v1 does not persist conversation state. This in-memory correlation ID maps directly to LangGraph's thread concept and enables future checkpointing, resume, and replay without retroactively adding identifiers.

3. Event-Typed Streaming

- The internal streaming layer must emit **typed events** rather than raw token strings. The v1 event types are: `token`, `tool_call`, `tool_result`, `status`, and `error`. Even though v1 consumers primarily use `token` events, this structure allows future agent orchestrators to emit additional event types (e.g., `plan_step`, `human_approval_needed`, `state_transition`) without breaking the streaming contract.

4. Multi-Step UX Readiness

- The UI layer must not assume that a single user action produces a single model response. The tool-call loop already breaks this assumption, but the UX components (progress indicators, output panels) must be designed to display **step-by-step progress** for multi-turn interactions. This ensures future agent workflows — which may involve planning, branching, and human-in-the-loop approvals — can surface their state in the existing UI framework.

2.5 Assumptions and Dependencies

Assumptions

ID	Assumption	Risk if Invalid
A-01	Customers have access to an OpenAI API-compatible service (Azure OpenAI, OpenAI, or enterprise gateway)	Must provide guidance on provider setup
A-02	Developer workstations can reach the AI provider endpoint (directly or via corporate proxy)	Must support proxy configuration
A-03	Target providers implement OpenAI API v1 specification	Azure OpenAI ✅, OpenAI ✅, most enterprise gateways ✅
A-04	IT administrators have mechanisms to deploy configuration to developer	Central config management is optional enhancement

ID	Assumption	Risk if Invalid
	machines (GPO, MDM, config management)	
A-05	VS Code and Eclipse support our extension's settings schema	Standard extension settings mechanisms exist

Dependencies (Internal Engineering)

Dependency	Owner	Status	Risk if Delayed
Configuration schema design	Extension Team	🟡 In Progress	Cannot finalize settings documentation
VS Code settings integration	VS Code Team	🟢 Complete	Basic extension settings work
Eclipse preferences integration	Eclipse Team	🟡 In Progress	Eclipse users cannot configure AI
Credential storage via SecretStorage	Platform Team	🟢 Complete (v0.7)	Credentials stored in OS-level secure storage via VS Code API
HTTP client with proxy support	Platform Team	🟢 Complete	Corporate proxy environments blocked
Central config profile system	Platform Team	🔴 Not Started (v1.1)	No managed configuration option

Customer-Side Prerequisites

Prerequisite	Customer Responsibility	We Provide
AI Provider Access	Provision Azure OpenAI or obtain OpenAI API access	Configuration examples per provider
API Credentials	Obtain API key from their provider	Secure storage guidance
Network Access		

Prerequisite	Customer Responsibility	We Provide
	Ensure workstations can reach provider endpoint	Proxy configuration documentation
(Optional) Config Distribution	Deploy settings to developer machines	Settings schema documentation

---

← Previous | Back to main | Next →

3. System Features (Functional Requirements)

EARS Syntax Legend

All requirements use the **Easy Approach to Requirements Syntax (EARS)** for clarity and testability:

Pattern	Example	When to Use
Ubiquitous	"The extension **shall**…"	Always applies
Event-driven	"**When** X occurs, the extension **shall**…"	Triggered by event
State-driven	"**While** in state X, the extension **shall**…"	Applies during state
Optional feature	"**Where** feature X is enabled, the extension **shall**…"	Conditional on configuration
Unwanted behavior	"**If** X occurs, the extension **shall**…"	Error handling

Priority Legend (MoSCoW): - **Must:** Critical for launch; extension is not viable without it - **Should:** Important but not critical; can launch without if necessary

- **Could:** Desirable; implement if time permits - **Won't (v1):** Explicitly excluded from this version

---

Requirement ID Migration Table

This table maps original requirement IDs to new domain-prefixed IDs:

Old ID	New ID	Domain	Old ID	New ID	Domain
REQ-01	REQ-CFG-001	Configuration	REQ-52	REQ-CRED-008	Credential
REQ-02	REQ-CFG-002	Configuration	REQ-52a	REQ-CRED-009	Credential
REQ-03	REQ-CFG-003	Configuration	REQ-53	REQ-CRED-010	Credential
REQ-04	REQ-CFG-004	Configuration	REQ-54	REQ-CRED-011	Credential
REQ-05	REQ-CFG-005	Configuration	REQ-12	REQ-API-001	API
REQ-06	REQ-CFG-006	Configuration	REQ-13	REQ-API-002	API
REQ-07	REQ-CRED-001	Credential	REQ-14	REQ-API-003	API
REQ-07a	REQ-CRED-002	Credential	REQ-15	REQ-API-004	API
REQ-08	REQ-CRED-003	Credential	REQ-16	REQ-API-005	API
REQ-09	REQ-CRED-004	Credential	REQ-17	REQ-API-006	API
REQ-10	REQ-CRED-005	Credential	REQ-93	REQ-API-007	API
REQ-11	REQ-CRED-006	Credential	REQ-94	REQ-API-008	API
REQ-55		Credential	REQ-79		Provider

Old ID	New ID	Domain	Old ID	New ID	Domain
	REQ-CRED-007			REQ-PROV-001	
REQ-80	REQ-PROV-002	Provider	REQ-72	REQ-TOOL-005	Tool
REQ-81	REQ-PROV-003	Provider	REQ-73	REQ-TOOL-006	Tool
REQ-82	REQ-PROV-004	Provider	REQ-74	REQ-TOOL-007	Tool
REQ-83	REQ-PROV-005	Provider	REQ-75	REQ-TOOL-008	Tool
REQ-68	REQ-PROV-006	Provider	REQ-77	REQ-TOOL-009	Tool
REQ-69	REQ-PROV-007	Provider	REQ-56	REQ-MCP-001	MCP
REQ-70	REQ-PROV-008	Provider	REQ-57	REQ-MCP-002	MCP
REQ-71	REQ-PROV-009	Provider	REQ-58	REQ-MCP-003	MCP
REQ-84	REQ-PROV-010	Provider	REQ-59	REQ-MCP-004	MCP
REQ-85	REQ-PROV-011	Provider	REQ-59a	REQ-MCP-005	MCP
REQ-18	REQ-UX-001	UX	REQ-59b	REQ-MCP-006	MCP
REQ-19	REQ-UX-002	UX	REQ-22	REQ-ENT-001	Enterprise
REQ-20	REQ-UX-003	UX	REQ-23	REQ-ENT-002	Enterprise
REQ-21	REQ-UX-004	UX	REQ-60	REQ-ENT-003	Enterprise
REQ-21a	REQ-UX-005	UX	REQ-61	REQ-ENT-004	Enterprise

Old ID	New ID	Domain	Old ID	New ID	Domain
REQ-76	REQ-UX-006	UX	REQ-62	REQ-ENT-005	Enterprise
REQ-76a	REQ-UX-007	UX	REQ-63	REQ-ENT-006	Enterprise
REQ-76b	REQ-UX-008	UX	REQ-89	REQ-LIFE-001	Lifecycle
REQ-77 → REQ-TOOL-009			REQ-90	REQ-LIFE-002	Lifecycle
REQ-78	REQ-UX-009	UX	REQ-99	REQ-LIFE-003	Lifecycle
REQ-28	REQ-PROMPT-001	Prompt			
REQ-29	REQ-PROMPT-002	Prompt			
REQ-30	REQ-PROMPT-003	Prompt			
REQ-31	REQ-PROMPT-004	Prompt			
REQ-32	REQ-PROMPT-005	Prompt			
REQ-86	REQ-PROMPT-006	Prompt			
REQ-87	REQ-PROMPT-007	Prompt			
REQ-88	REQ-PROMPT-008	Prompt			
REQ-33	REQ-JSON-001	JSON			
REQ-34	REQ-JSON-002	JSON			

Old ID	New ID	Domain	Old ID	New ID	Domain
REQ-35	REQ-JSON-003	JSON			
REQ-36	REQ-JSON-004	JSON			
REQ-37	REQ-TOOL-001	Tool			
REQ-38	REQ-TOOL-002	Tool			
REQ-39	REQ-TOOL-003	Tool			
REQ-40	REQ-TOOL-004	Tool			
REQ-41	REQ-TOOL-010	Tool	(Req-41 maps to 010)		

3.1 Configuration Management

ID	Priority	Requirement (EARS Format)
REQ-CFG-001	Must	**The extension shall** read configuration from IDE-native settings (VS Code `settings.json`, Eclipse preferences).
REQ-CFG-002	Must	**The extension shall** support configuration in both JSON and YAML formats where the IDE permits.
REQ-CFG-003	Must	**The extension shall** allow configuration of `base_url` to any OpenAI API-compatible endpoint.
REQ-CFG-004	Must	**The extension shall** support model ID aliasing, mapping user-friendly names to provider model identifiers.
REQ-CFG-005	Must	

ID	Priority	Requirement (EARS Format)
		The extension shall support configuration of custom headers (e.g., `api-version` for Azure OpenAI).
REQ-CFG-006	Should	**When** both user and workspace settings exist, **the extension shall** merge them with **workspace settings taking precedence** per VS Code convention (user settings as fallback).

3.2 Credential Management (BYOK)

ID	Priority	Requirement (EARS Format)
REQ-CRED-001	Must	**The extension shall** use VS Code `SecretStorage` API as the primary credential storage mechanism; **when** SecretStorage is unavailable (e.g., headless/container environments), **the extension shall** fall back to OS credential stores (macOS Keychain, Windows Credential Manager, Linux Secret Service) with a warning to the user about reduced security.
REQ-CRED-002	Must	**When** both SecretStorage and OS credential stores are unavailable, **the extension shall** disable AI features requiring authentication and display a persistent warning: “Credential storage unavailable. AI features disabled. Use environment variables (OPENAI_API_KEY) as workaround.”
REQ-CRED-003	Must	**The extension shall** support retrieving API keys from environment variables.
REQ-CRED-004	Should	**The extension shall** support API key entry via IDE-native secret input (masked input fields).
REQ-CRED-005	Must	**The extension shall not** store API keys in plaintext in settings files by default.
	Must	

ID	Priority	Requirement (EARS Format)
REQ-CRED-006		**The extension shall** never include API keys in logs, error reports, or telemetry.
REQ-CRED-007	Should	**The extension shall** provide a first-run “Set API Key” command that writes the key to `SecretStorage` and confirms success to the user.
REQ-CRED-008	Must	**The extension shall** use the VS Code `SecretStorage` API (`context.secrets`) to persist API keys locally on the user’s machine.
REQ-CRED-009	Must	**The extension shall** provide a “Delete API Key” command that removes the stored credential from `SecretStorage` and confirms the deletion to the user.
REQ-CRED-010	Must	**The extension shall not** store API keys in `globalState`, `workspaceState`, or standard JSON settings files.
REQ-CRED-011	Must	**The extension shall** support dynamic Header Injection, populating authentication headers (e.g., `Authorization`, `X-Api-Key`) from `SecretStorage` at request time.

3.3 API Integration

ID	Priority	Requirement (EARS Format)
REQ-API-001	Must	**The extension shall** implement the OpenAI Chat Completions API (`/v1/chat/completions`).
REQ-API-002	Must	**The extension shall** support streaming responses via Server-Sent Events (SSE).
REQ-API-003	Must	**When** a request includes `tools` or `functions` parameters, **the extension shall** pass them unmodified to the provider.
REQ-API-004	Must	

ID	Priority	Requirement (EARS Format)
		When `response_format: { "type": "json_object" }` is specified, **the extension shall** include it in the request.
REQ-API-005	Should	**The extension shall** support the Images API (`/v1/images/generations`) for image generation features.
REQ-API-006	Must	**The extension shall** support HTTP/HTTPS proxy configuration via IDE proxy settings and/or environment variables.
REQ-API-007	Must	**The extension shall** enforce network timeouts: 10 seconds for connection establishment, 60 seconds for streaming reads, 30 seconds for non-streaming reads.
REQ-API-008	Must	**When** a provider response exceeds 1MB, **the extension shall** truncate the response and display a warning: “Response truncated at 1MB. Consider using streaming or reducing request scope.”

3.4 Provider Management

ID	Priority	Requirement (EARS Format)
REQ-PROV-001	Must	**The extension shall** support multiple concurrent chat sessions without blocking or interference.
REQ-PROV-002	Should	**When** multiple requests are initiated simultaneously, **the extension shall** queue them and enforce per-provider rate limits to prevent throttling errors.
REQ-PROV-003	Should	**The extension shall** detect provider support for vision/multimodal inputs via `/v1/models` endpoint or provider-specific capability metadata.
REQ-PROV-004	Should	**When** a provider does not support a requested feature (e.g., tool calling, vision), **the extension shall** gracefully degrade by disabling the feature and notifying the user.

ID	Priority	Requirement (EARS Format)
REQ-PROV-005	Should	**The extension shall** cache provider capability information for the duration of the session to minimize redundant API calls.
REQ-PROV-006	Must	**The extension shall** treat the configured `baseUrl` as the complete base path to which API-specific paths (e.g., `/chat/completions`) are appended. For Azure OpenAI, users must include the full deployment path in `baseUrl`.
REQ-PROV-007	Must	**The extension shall** apply provider-specific behaviors (auth header name, default headers) based on the configured `type` field as specified in the Provider Type Behaviors table.
REQ-PROV-008	Must	**The extension shall** allow users to override the default auth header name via the `authHeaderName` configuration field.
REQ-PROV-009	Must	**The extension shall** support three authentication modes via the `authMode` field: `"byok"` (SecretStorage), `"sso"` (vscode.authentication), and `"none"` (no authentication).
REQ-PROV-010	Should	**The extension shall** support configuration of multiple named provider profiles (e.g., “dev-azure”, “prod-openai”) in user or workspace settings.
REQ-PROV-011	Should	**The extension shall** provide a UI command or quick pick to switch the active provider profile at runtime.

3.5 User Experience

ID	Priority	Requirement (EARS Format)
REQ-UX-001	Should	**When** configuration is incomplete, **the extension shall** prompt the user with guided setup.

ID	Priority	Requirement (EARS Format)
REQ-UX-002	Should	**The extension shall** provide a “Test Connection” command to validate provider connectivity.
REQ-UX-003	Must	**When** an API error occurs, **the extension shall** display a user-friendly error message with troubleshooting guidance.
REQ-UX-004	Must	**The extension shall** indicate streaming progress during AI response generation.
REQ-UX-005	Should	**When** a user is waiting for an AI response, **the extension shall** display a progress indicator showing elapsed time and estimated completion if streaming has not yet started.
REQ-UX-006	Must	**The extension shall** support user-initiated cancellation of in-progress streaming requests via standard UI affordances (e.g., Escape key, Stop button).
REQ-UX-007	Should	**When** a streaming request is cancelled, **the extension shall** abort the HTTP request, clear partial UI state, and display a confirmation message (e.g., “Request cancelled”).
REQ-UX-008	Should	**When** a request is in progress, **the extension shall** display a cancellable progress indicator with elapsed time.
REQ-UX-009	Must	**The extension shall** sanitize provider error messages before displaying to users or writing to logs, removing internal URLs, resource identifiers, and any string matching common API key patterns.
REQ-UX-010	Must	**The extension shall** support keyboard-only operation: all AI commands accessible via Command Palette (VS Code) or menu shortcuts, no mouse-only features.

3.6 Prompt Construction & Context

ID	Priority	Requirement (EARS Format)
REQ-PROMPT-001	Must	**The extension SDK shall** support all OpenAI message roles: `system`, `user`, `assistant`, and `tool`.
REQ-PROMPT-002	Must	**The extension SDK shall** provide a programmatic API for constructing message arrays without requiring user interaction.
REQ-PROMPT-003	Should	**The extension SDK shall** support prompt templates with variable substitution for reusable, parameterized prompts.
REQ-PROMPT-004	Should	**When** constructing prompts, **the extension SDK shall** provide utilities to estimate token counts before sending requests.
REQ-PROMPT-005	Should	**The extension SDK shall** support context prioritization, allowing developers to specify which context to retain when truncating for token limits.
REQ-PROMPT-006	Should	**When** the constructed prompt exceeds 90% of the model's known context window, **the extension shall** warn the user before sending the request.
REQ-PROMPT-007	Should	**When** a provider returns a token limit error (e.g., HTTP 400 with `context_length_exceeded`), **the extension shall** automatically truncate the oldest non-system messages and retry once.
REQ-PROMPT-008	Should	**When** automatic truncation occurs, **the extension shall** notify the user that context was reduced and log which messages were removed.

3.7 Structured Output

ID	Priority	Requirement (EARS Format)
REQ-JSON-001	Must	**The extension SDK shall** support the `response_format: { type: "json_object" }` parameter for JSON mode.
REQ-JSON-002	Must	**When** JSON mode is requested, **the extension SDK shall** validate that the system prompt instructs the model to produce JSON.
REQ-JSON-003	Should	**The extension SDK shall** provide typed response parsing with runtime validation against a provided schema.
REQ-JSON-004	Should	**When** JSON parsing fails, **the extension SDK shall** provide a clear error indicating the parsing failure and include the raw response for debugging.

3.8 Function Calling (Tools)

ID	Priority	Requirement (EARS Format)
REQ-TOOL-001	Must	**The extension SDK shall** support defining tools/functions with JSON Schema parameter definitions.
REQ-TOOL-002	Must	**The extension SDK shall** support all `tool_choice` options: `auto`, `required`, `none`, and specific function selection.
REQ-TOOL-003	Must	**When** the model returns `tool_calls`, **the extension SDK shall** provide the parsed function name and arguments.
REQ-TOOL-004	Must	**The extension SDK shall** support sending tool results back to the model via the `tool` role message.
REQ-TOOL-005	Must	

ID	Priority	Requirement (EARS Format)
		The extension's tool-call orchestration loop **shall** terminate after a configurable maximum number of iterations (default: 25) to prevent infinite loops.
REQ-TOOL-006	Must	**The extension shall** enforce a configurable timeout (default: 30 seconds) for individual tool executions to prevent indefinite hangs.
REQ-TOOL-007	Must	**When** a tool is classified as destructive (e.g., `run_terminal_command`), **the extension shall** require explicit user approval before execution if `requireApprovalForDestructiveTools` is enabled.
REQ-TOOL-008	Must	**The extension's** internal tool-call orchestrator **shall** invoke tools via direct function calls (not via the MCP protocol) to avoid serialization overhead for its own LLM interactions.
REQ-TOOL-009	Must	**When** a tool execution returns a result exceeding 10,000 characters, **the extension shall** truncate the result and append a notice (e.g., "[Output truncated; chars omitted]") before sending it to the model.
REQ-TOOL-010	Should	**The extension SDK shall** provide a tool execution loop helper that handles multi-turn tool calling until completion.

3.9 Model Context Protocol (MCP)

ID	Priority	Requirement (EARS Format)
REQ-MCP-001	Must	**The extension shall** register as an MCP Server using the `vscode.lm.registerMcpServerDefinitionProvider` API.
REQ-MCP-002	Must	**The extension shall** expose extension-specific capabilities (e.g., `search_codebase`, `run_unit_tests`, `read_file_context`) as MCP Tools with JSON Schema parameter definitions.

ID	Priority	Requirement (EARS Format)
REQ-MCP-003	Must	**The extension shall** act as the execution layer ("the hands"), receiving tool-call requests from the cloud-hosted model and executing them via the VS Code Extension API.
REQ-MCP-004	Must	**The extension shall** validate and sandbox MCP tool executions to prevent unintended side effects on the user's workspace.
REQ-MCP-005	Must	**The extension shall** classify MCP tools by risk level (`safe`, `read-only`, `write-only`, `destructive`) and maintain an allowlist of external MCP hosts authorized to invoke each tool category.
REQ-MCP-006	Must	**When** the workspace is untrusted (per VS Code Workspace Trust API), **the extension shall** disable all tool execution except read-only context retrieval from open files.

3.10 Enterprise Graduation

ID	Priority	Requirement (EARS Format)
REQ-ENT-001	Won't (v1)	**The extension shall** support loading configuration profiles from a central management endpoint.
REQ-ENT-002	Won't (v1)	**When** managed configuration is deployed, **the extension shall** allow IT to lock certain settings from user override.
REQ-ENT-003	Must	**When** `BASE_URL` points to an enterprise gateway, **the extension shall** support SSO via the `vscode.authentication` provider, exchanging a VS Code session token for a gateway bearer token.
REQ-ENT-004	Must	**The extension shall** include an `X-Request-Id` header (UUID v4) in all outgoing LLM calls to enable end-to-end request tracing.
REQ-ENT-005	Must	**The extension shall** support a zero-code-change transition from direct SaaS access (BYOK) to enterprise-governed access (AI Gateway + SSO).

ID	Priority	Requirement (EARS Format)
REQ-ENT-006	Must	**The extension shall** respect VS Code's built-in proxy settings and `HTTP_PROXY` / `HTTPS_PROXY` environment variables for corporate network environments.

3.11 Extension Lifecycle

ID	Priority	Requirement (EARS Format)
REQ-LIFE-001	Must	**When** the extension is deactivated or reloaded, **the extension shall** abort in-progress requests, close open HTTP connections, and dispose of event listeners to prevent memory leaks.
REQ-LIFE-002	Should	**The extension shall** implement graceful degradation during IDE shutdown: streaming requests shall be cancelled with a timeout of 5 seconds before forceful termination.
REQ-LIFE-003	Must	**The extension shall** activate on the following events: (1) `onStartupFinished` for background MCP Server registration, (2) `onCommand:ourProduct.ai.*` for user-invoked AI commands, (3) `onView:ourProductAiChat` if a dedicated AI view is contributed. The extension shall complete activation within 2 seconds (per NFR).

Security & Privacy Requirements

Note: Security requirements have been reclassified into domain-specific non-functional requirements. See [Non-Functional Requirements](#) §5.3 Security Requirements.

Prompt Injection Mitigation:

ID	Priority	Requirement (EARS Format)
REQ-SEC-001	Must	**When** constructing prompts with user-provided content, **the extension SDK shall** clearly delimit user content to reduce prompt injection risk.

Future-Proofing (Architectural Hedges)

These requirements support future agent framework integration without requiring v1 feature implementations:

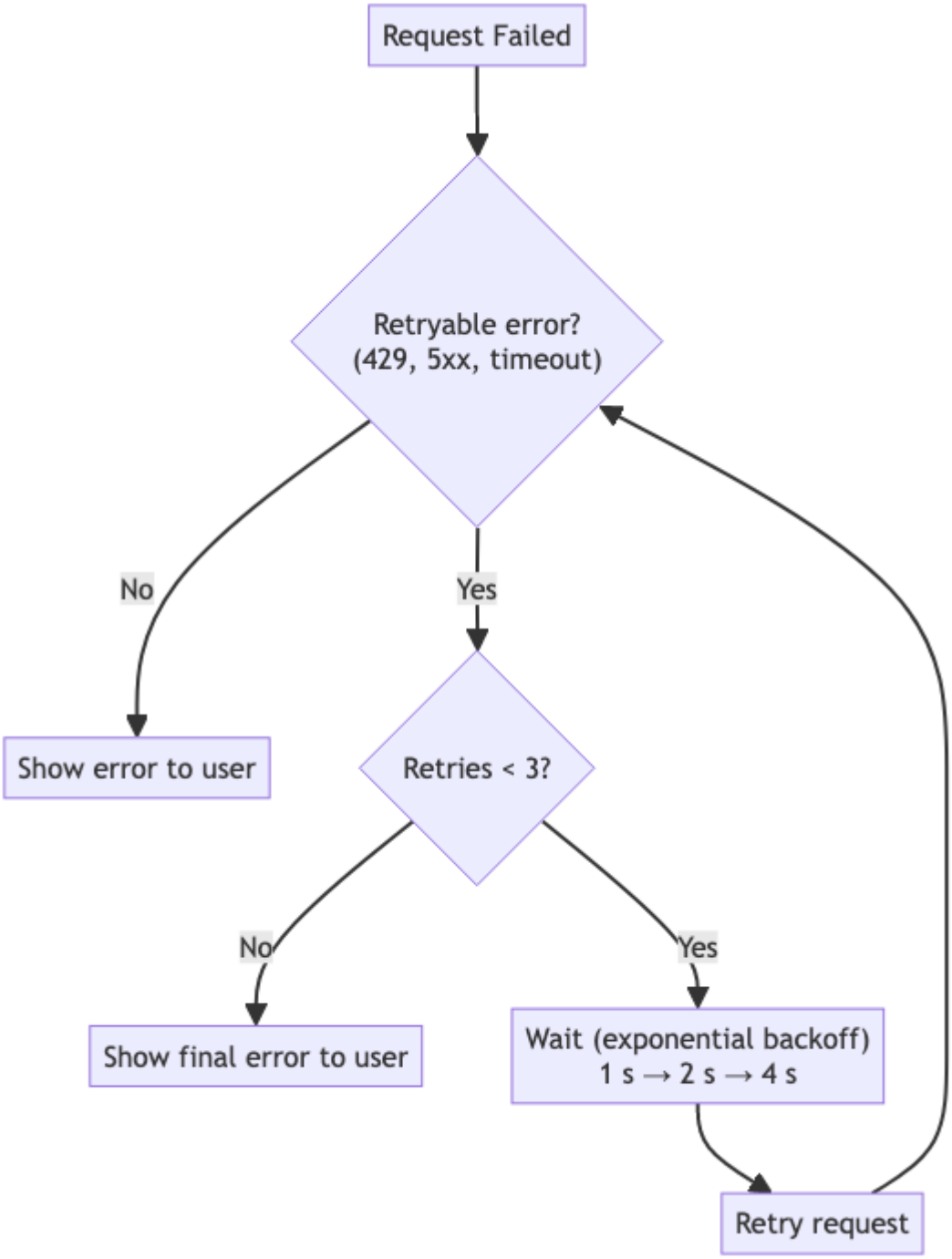
ID	Priority	Requirement (EARS Format)
REQ-ARCH-001	Must	**The extension shall** encapsulate tool-call orchestration behind a `ToolOrchestrator` interface, allowing the iterative tool-call loop to be replaced with an alternative orchestration backend without modifying other extension components.
REQ-ARCH-002	Must	**The extension shall** assign a `thread_id` (UUID v4) to every conversation or multi-turn interaction, available in-memory for the duration of the interaction.
REQ-ARCH-003	Must	**The extension's** internal streaming layer **shall** emit typed events (minimally: `token`, `tool_call`, `tool_result`, `status`, `error`) rather than raw token strings, to support future event types without breaking the streaming contract.
REQ-ARCH-004	Should	**The extension's** UI components for AI interactions **shall** support displaying step-by-step progress for multi-turn tool-call sequences, not assuming a single model response per user action.

Error Handling Context

Provider Error Responses

Provider Error	Extension Behavior	User Feedback
401 Unauthorized	No retry	“Authentication failed. Check your API key.” + link to settings
403 Forbidden	No retry	“Access denied. Your API key may lack required permissions.”
429 Rate Limited	Retry with backoff (respects `Retry-After`)	“Rate limited. Retrying…” or “Rate limit exceeded. Try again later.”
400 Bad Request	No retry	“Request error: {sanitized provider message}” (see REQ-UX-009 for sanitization rules)
500 Internal Error	Retry up to 3x with exponential backoff	“Provider error. Retrying…” or “Provider unavailable.”
502/503 Unavailable	Retry up to 3x	“Provider temporarily unavailable. Retrying…”
Timeout	Retry once	“Request timed out. Retrying…” or “Request timed out. Check network.”
Network Error	Check connectivity	“Cannot reach AI provider. Check your network connection.”

Retry Strategy



← Previous | Back to main | Next →

4. External Interface Requirements

4.1 User Interfaces

VS Code Settings (`settings.json`)

Configuration uses VS Code's native settings system. Our extension contributes settings under a namespace (e.g., `ourProduct.ai`). This approach aligns with VS Code best practices and allows familiar configuration patterns.

Example: User or Workspace `settings.json`

```
{
  // AI Provider Configuration
  "ourProduct.ai.provider": {
    // Provider endpoint (required)
    "baseUrl": "https://mycompany.openai.azure.com/openai/
deployments/gpt-4",

    // Provider type for provider-specific behaviors (see
Provider Type Behaviors table below)
    "type": "azure-openai",  // "azure-openai" | "openai" |
"anthropic" | "ollama" | "openshift-ai" | "bedrock" |
"compatible"

    // Authentication mode: "byok" (Bring Your Own Key via
SecretStorage), "sso" (Enterprise SSO via
vscode.authentication), or "none" (no auth)
    "authMode": "byok",  // "byok" | "sso" | "none"

    // Override auth header name (optional; defaults per
provider type in Provider Type Behaviors table)
    "authHeaderName": "Authorization",  // e.g.,
"Authorization", "X-Api-Key", "api-key"

    // Additional headers (e.g., Azure API version)
    "headers": {
      "api-version": "2024-02-15-preview"
    },
```

```
    // Default model (can be overridden per feature)
    "defaultModel": "gpt-4-turbo"
  },

  // Model aliases - map friendly names to provider model IDs
  "ourProduct.ai.models": {
    "default": "gpt-4-turbo",
    "fast": "gpt-35-turbo",
    "reasoning": "gpt-4-turbo"
  },

  // Credential source (API key retrieved separately for
security)
  "ourProduct.ai.credentials": {
    // Where to get the API key
    "source": "keychain",  // "keychain" | "environment" |
"prompt"

    // Environment variable name (if source is "environment")
    "environmentVariable": "OPENAI_API_KEY",

    // Keychain/Credential Manager entry name (if source is
"keychain")
    "keychainEntry": "ourProduct-ai-apikey"
  },

  // Feature-specific settings
  "ourProduct.ai.codeCompletion": {
    "enabled": true,
    "model": "fast",  // Uses model alias
    "maxTokens": 500
  },

  "ourProduct.ai.chat": {
    "enabled": true,
    "model": "default",
    "maxTokens": 4096,
    "temperature": 0.7
  },

  // Tool execution safety limits
```

```
  "ourProduct.ai.tooling": {
    // Maximum iterations for tool-call orchestration loop
(circuit breaker)
    "maxLoopIterations": 25,

    // Tool execution timeout in seconds
    "toolTimeout": 30,

    // Require user approval for destructive tools
(run_terminal_command, etc.)
    "requireApprovalForDestructiveTools": true
  },

  // Network settings
  "ourProduct.ai.network": {
    // Use VS Code's proxy settings, or override here
    "useSystemProxy": true,

    // Custom CA certificate for enterprise PKI
    "caCertificatePath": "/path/to/corporate-ca.crt",

    // Request timeout in seconds
    "timeout": 60
  },

  // Privacy settings
  "ourProduct.ai.telemetry": {
    "enabled": false  // Opt-in only
  }
}
```

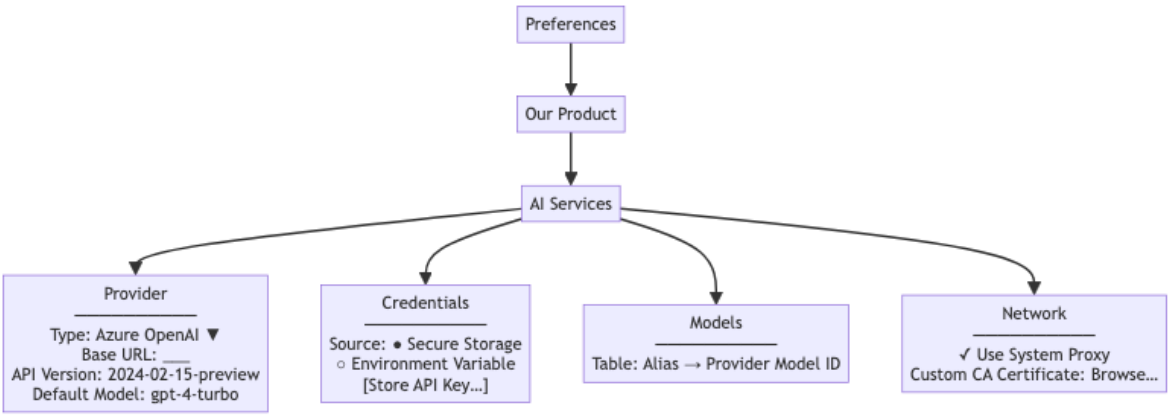
Eclipse Preferences

Eclipse uses its preference system. Configuration is stored in workspace or user preferences. The extension shall provide a preference page under the product namespace accessible via **Window > Preferences > [Product Name](#) > AI Services**.

Implementation Requirements: - Preference page shall use Eclipse JFace preference framework with `FieldEditorPreferencePage` - Provider type shall be implemented as `ComboFieldEditor` with enum validation - API keys shall use `StringFieldEditor` with `echo` char set to `'*'` for masking - Base URL shall use

`StringFieldEditor` with URL validation - Credentials shall be stored in Eclipse Secure Storage (`ISecurePreferences` API) under node `com.ourproduct.ai/credentials` - Preference changes shall trigger `IPropertyChangeListener` notifications to refresh active connections

Example: Preference Page Structure



Configuration Examples by Provider

Azure OpenAI

```
{
  "ourProduct.ai.provider": {
    "type": "azure-openai",
    "baseUrl": "https://{resource-name}.openai.azure.com/
openai/deployments/{deployment-name}",
    "authMode": "byok",
    "headers": {
      "api-version": "2024-02-15-preview"
    }
  }
}
```

Note: The extension appends `/chat/completions` to the `baseUrl`, so the full request URL becomes:

```
https://{resource-name}.openai.azure.com/openai/deployments/
{deployment-name}/chat/completions?api-version=2024-02-15-
preview
```

OpenAI (Direct)

```
{
  "ourProduct.ai.provider": {
    "type": "openai",
    "baseUrl": "https://api.openai.com/v1",
    "defaultModel": "gpt-4-turbo"
  }
}
```

Enterprise Gateway

```
{
  "ourProduct.ai.provider": {
    "type": "compatible",
    "baseUrl": "https://ai-gateway.internal.mycompany.com/
v1",
    "authMode": "sso",  // Use enterprise SSO instead of BYOK
    "headers": {
      "X-Department": "engineering",
      "X-Project-ID": "project-123"
    }
  }
}
```

Ollama (Self-Hosted)

```
{
  "ourProduct.ai.provider": {
    "type": "ollama",
    "baseUrl": "http://localhost:11434/v1",
    "defaultModel": "llama3"
  }
}
```

Note: For remote Ollama instances, use the appropriate hostname/IP (e.g., `http://ollama-server.internal:11434/v1`). No API key required for local instances; authentication can be configured if Ollama is secured.

Credential Storage Options

Method	Platform	How It Works
macOS Keychain	macOS	Stored in system Keychain; accessed via native APIs
Windows Credential Manager	Windows	Stored in Windows Credential Manager
Linux Secret Service	Linux	Stored via libsecret (GNOME Keyring, KWallet)
Environment Variable	All	Read from specified env var at runtime
Interactive Prompt	All	User prompted on first use; stored in credential store

Configuration Validation

The extension validates configuration and provides helpful feedback:

Issue	User Feedback
Missing `baseUrl`	“AI provider not configured. Open Settings to configure.”
Invalid URL format	“Invalid provider URL. Expected format: https://…”
Missing API key	“API key not found. Run ‘Configure AI API Key’ command.”
Connection failed	“Cannot connect to AI provider. Check URL and network settings.”
Authentication failed	“API key rejected by provider. Verify your API key is correct.”

4.2 Hardware Interfaces

Not applicable. The extension runs within the IDE process and does not directly interface with hardware. All hardware interaction (network, storage) is mediated through IDE APIs and the operating system.

4.3 Software Interfaces

Supported Providers

Provider	Compatibility	Configuration Notes
Azure OpenAI	✅ Native	Set `base_url` to deployment; add `api-version` header
OpenAI	✅ Native	Default; just provide API key
Enterprise Gateways	✅ Native	Point `base_url` to gateway; gateway handles upstream
Ollama	✅ Native	Self-hosted; point `base_url` to local instance (e.g., `http://localhost:11434/v1`)
Anthropic Claude	✅ Via OpenAI-compatible endpoint	Set `base_url` to `https://api.anthropic.com/v1/`; BYOK with Anthropic API key. Uses Anthropic's OpenAI-compatible surface — no adapter or translation layer needed.
Red Hat OpenShift AI	✅ Via OpenAI-compatible endpoint	Set `base_url` to the OpenShift AI model serving route (e.g., `https://<model-route>.apps.<cluster>/v1`). Uses KServe / vLLM runtime which exposes an OpenAI-compatible inference API. BYOK with the serving endpoint's bearer token.
AWS Bedrock	⚠️ Conditional	Supported when accessed through Bedrock's OpenAI-compatible

Provider	Compatibility	Configuration Notes
		endpoint or an enterprise AI gateway that exposes an OpenAI-compatible surface. Native Bedrock SigV4 auth is **not** supported in v1; customers requiring direct Bedrock access without a gateway should use the gateway graduation path.

Provider Type Behaviors

The `type` field in the provider configuration controls provider-specific behaviors:

Provider Type	Auth Header	Path Construction	Default Headers	Notes
`azure-openai`	`api-key`	`baseUrl` + `/chat/completions` + `?api-version=X`	`api-key: <from SecretStorage>`	Requires `api-version` in headers or query params
`openai`	`Authorization`	`baseUrl` + `/chat/completions`	`Authorization: Bearer <from SecretStorage>`	Standard OpenAI format
`anthropic`	`x-api-key`	`baseUrl` + `/chat/completions`	`x-api-key: <from SecretStorage>` `anthropic-version: 2023-06-01`	Uses Anthropic's OpenAI-compatible endpoint
`ollama`	None (optional)	`baseUrl` + `/chat/completions`	None	No auth by default; supports optional Bearer token if Ollama is secured

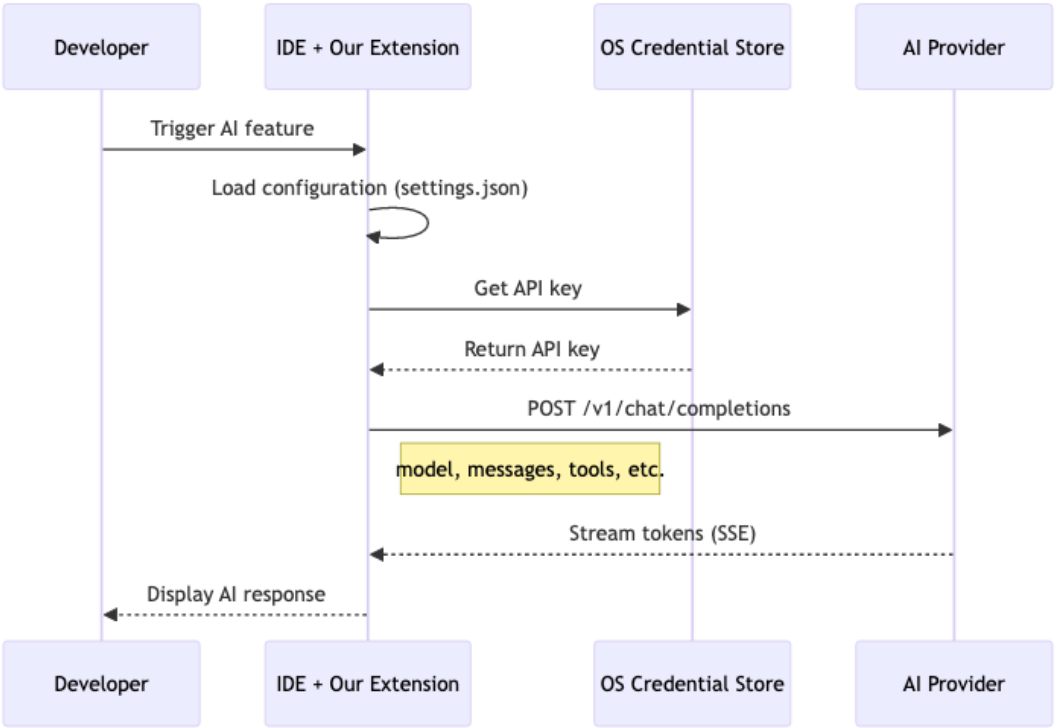
Provider Type	Auth Header	Path Construction	Default Headers	Notes
`openshift-ai`	`Authorization`	`baseUrl` + `/chat/completions`	`Authorization: Bearer <from SecretStorage>`	Uses Kubernetes service account or OAuth token
`bedrock`	N/A (v1.1)	N/A	N/A	Native SigV4 auth deferred; use via gateway or OpenAI-compatible endpoint
`compatible`	Configurable via `authHeaderName`	`baseUrl` + `/chat/completions`	User-specified via `authHeaderName` and `authMode`	Generic fallback for any OpenAI-compatible provider

Configurability: Users can override the default auth header using the `authHeaderName` field in the provider configuration. When `authMode: "sso"` is set, the `vscode.authentication` provider is used instead of `SecretStorage`.

4.4 Communications Interfaces

Request Flow

The extension communicates with AI providers using HTTPS and the OpenAI Chat Completions API format:



Network Protocols

Protocol	Usage	Requirements
HTTPS	All provider communication	TLS 1.2 or higher; custom CA certificates supported
Server-Sent Events (SSE)	Streaming responses	Standard `text/event-stream` format
HTTP/HTTPS Proxy	Corporate network environments	Respect VS Code proxy settings and `HTTP_PROXY` / `HTTPS_PROXY` environment variables

Security

Concern	How We Address
API Key Storage	Support OS credential stores (Keychain, Credential Manager, Secret Service); discourage plaintext in settings
Credential Leakage	API keys never logged; not included in error reports
Network Security	

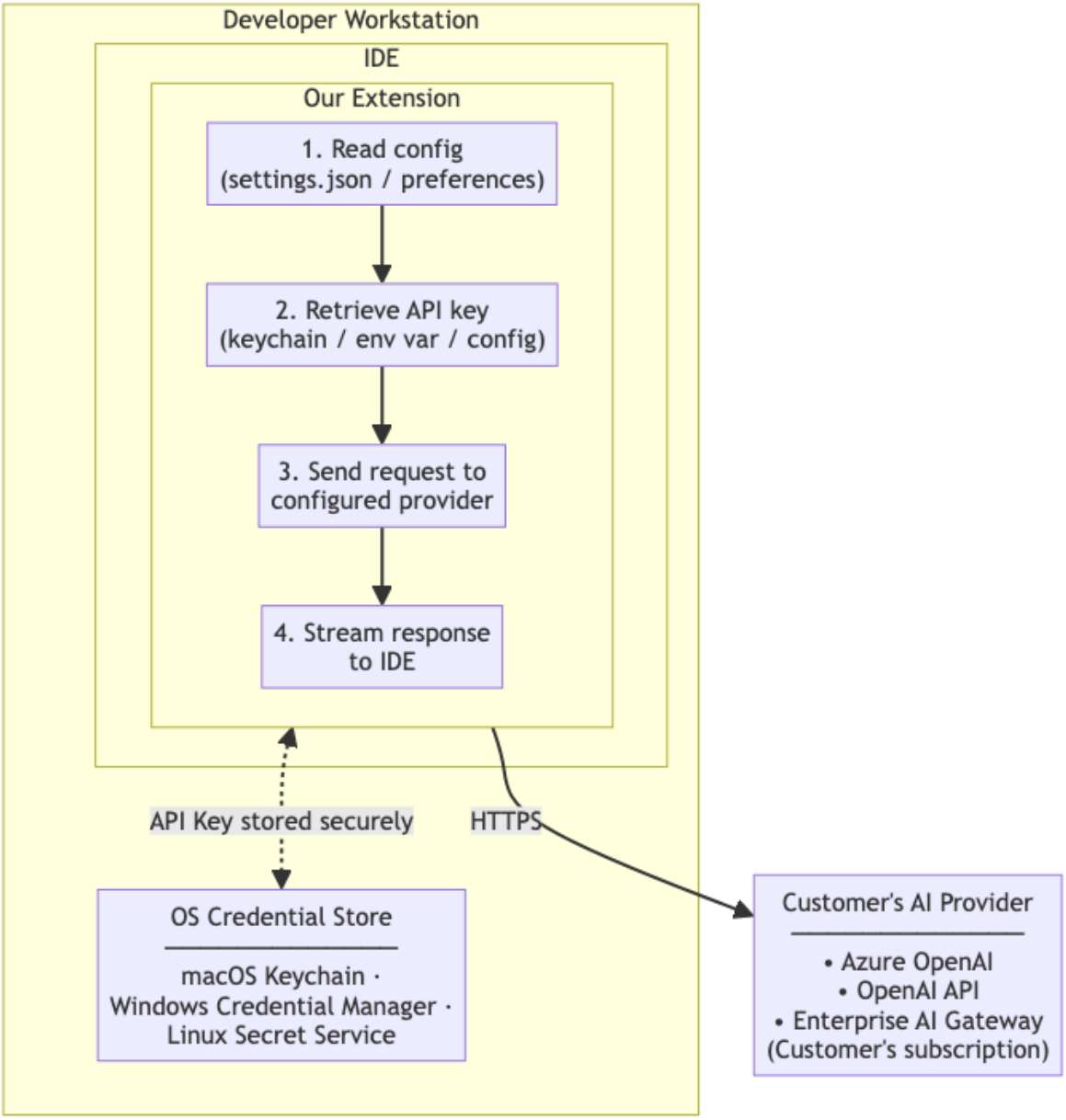
Concern	How We Address
	HTTPS required for provider communication; proxy support for corporate networks
Configuration Security	Settings can be scoped to user or workspace; sensitive fields marked appropriately

Enterprise Security Patterns

Pattern	Support
Corporate Proxy	HTTP_PROXY, HTTPS_PROXY environment variables; IDE proxy settings
Certificate Pinning	Custom CA certificate configuration for enterprise PKI
Credential Rotation	Credentials fetched fresh; no caching that would delay rotation
Audit Logging	Provider-side logging (Azure OpenAI logs, OpenAI usage dashboard)

Data Flow

Our extension runs on the developer's workstation and communicates directly with the customer's configured AI provider:



What We (the Vendor) See

Data Type	Do We See It?	Notes
Prompts / Code	❌ No	Sent directly from workstation to customer's provider
Completions	❌ No	Returned directly from provider to workstation

Data Type	Do We See It?	Notes
API Keys	❌ No	Stored on customer's machine; never transmitted to us
Configuration	❌ No	Local to customer's machines
Usage Telemetry	⚠️ Only if opted-in	Optional, anonymized telemetry for product improvement

---

← Previous | Back to main | Next →

5. Non-Functional Requirements

5.1 Performance Requirements

ID	Category	Requirement	Target	Measurement
PERF-001	Latency	Extension overhead (excluding provider time)	< 50ms	Performance profiling
PERF-002	Responsiveness	Time to first token displayed	< 500ms after provider responds	User testing
PERF-003	Bundle Size	Extension package size	< 10MB (VS Code), < 15MB (Eclipse with dependencies)	Build artifact size check

ID	Category	Requirement	Target	Measurement
PERF-004	CPU	Idle CPU usage	< 5% when no AI requests active	Profiling
PERF-005	CPU	Active CPU usage	< 20% during streaming response	Profiling
PERF-006	Memory	Extension memory footprint	< 100MB	IDE memory profiling
PERF-007	Startup	Extension activation time	< 2 seconds	Activation timing

Additional Performance Constraints:

- **Streaming latency:** Time from SSE first byte to UI display must be < 500ms
- **Network timeouts:** See REQ-API-007 (10s connection, 60s streaming, 30s non-streaming)
- **Response size limits:** See REQ-API-008 (1MB truncation with warning)

5.2 Safety Requirements

Not applicable. This software does not operate in safety-critical environments. The extension provides AI-assisted development tools and does not control any safety-critical systems.

Risk Mitigation:

While not safety-critical, the extension implements safeguards to prevent unintended workspace modifications:

- Tool execution requires explicit approval for destructive operations (REQ-TOOL-007)
- Workspace Trust integration disables tool execution in untrusted workspaces (REQ-MCP-006)
- Tool result truncation prevents context window overflow (REQ-TOOL-009)
- Circuit breaker prevents infinite tool-call loops (REQ-TOOL-005)

5.3 Security Requirements

ID	Category	Requirement	Implementation
SEC-001	Encryption	All provider communication must use HTTPS	TLS 1.2 or higher required (REQ-CRED-001)
SEC-002	PKI Support	Support custom CA certificates for enterprise PKI environments	Custom CA certificate configuration (REQ-ENT-006)
SEC-003	Credential Protection	API keys never logged or included in error reports	Credential sanitization (REQ-CRED-006, REQ-UX-009)
SEC-004	Credential Storage	API keys stored in OS-level secure storage (SecretStorage)	Never in plaintext settings files (REQ-CRED-005, REQ-CRED-010)
SEC-005	Prompt Injection	User content clearly delimited to reduce injection risk	Content delimitation in prompt construction (REQ-SEC-001)

Additional Security Constraints:

- **No data persistence:** Prompts, completions, or conversation history not persisted to disk unless explicitly enabled for a feature
- **SecretStorage API unavailable degradation:** When SecretStorage unavailable, fall back to OS credential stores with warning; when both unavailable, disable AI features and display persistent warning (REQ-CRED-002)
- **Error message sanitization:** Provider error messages sanitized before display, removing internal URLs, resource identifiers, and API key patterns (REQ-UX-009)
- **Workspace Trust integration:** Tool execution disabled in untrusted workspaces; only read-only context from open files permitted (REQ-MCP-006)

5.4 Software Quality Attributes

Reliability

ID	Attribute	Requirement	Target
QUAL-001	Error Recovery	Automatic retry with exponential backoff for transient errors	3 retries max for 429, 5xx, timeout (see error handling strategy)
QUAL-002	Graceful Degradation	Continue operation when provider capabilities unavailable	Feature-specific fallbacks (REQ-PROV-004)
QUAL-003	Circuit Breaker	Prevent infinite loops in tool-call orchestration	Max 25 iterations (REQ-TOOL-005)
QUAL-004	Request Cancellation	Support user-initiated cancellation of in-progress requests	Clean abort with confirmation (REQ-UX-006, REQ-UX-007)

Accessibility

ID	Attribute	Requirement	Target
A11Y-001	Keyboard Navigation	All AI commands accessible via keyboard shortcuts	No mouse-only features (REQ-UX-010)
A11Y-002	Screen Reader Support	Status changes announced to screen readers	ARIA live regions for “Thinking…”, “Generating…”, “Complete” notifications

ID	Attribute	Requirement	Target
A11Y-003	Focus Management	Predictable focus flow during streaming; status changes announced	Accessibility testing
A11Y-004	High Contrast	UI elements visible in high contrast mode	Theme compatibility testing
A11Y-005	Zoom Support	UI remains usable at 200% zoom	VS Code zoom testing

Implementation: See REQ-UX-010 (keyboard operation) and the accessibility requirements for detailed specifications.

Telemetry & Observability

ID	Attribute	Requirement	Implementation
TELEM-001	Opt-in Only	No telemetry without explicit user consent	Default disabled; user must enable via settings
TELEM-002	Event Schema	Structured event types with anonymized metadata	Defined event types: `ai.request.started`, `ai.request.completed`, `ai.request.failed`, `ai.tool.executed`
TELEM-003	PII Exclusion	No prompts, API keys, file paths in telemetry payloads	Data sanitization before transmission
TELEM-004	Request Tracing	X-Request-Id header on all LLM requests	UUID v4 header for end-to-end observability (REQ-ENT-004)

Telemetry Event Schema Example:

```
interface TelemetryEvent {
  event_type: "ai.request.started" | "ai.request.completed"
| "ai.request.failed" | "ai.tool.executed";
  timestamp: string;  // ISO 8601
```

```
  session_id: string;  // UUID v4 (session-scoped, not
persisted)
  provider_type: "azure-openai" | "openai" | "anthropic" |
"ollama" | "compatible";
  model_family: string;  // e.g., "gpt-4", "gpt-3.5" (no
specific deployment ID)
  feature: string;  // e.g., "code_completion", "chat",
"code_review"
  duration_ms?: number;
  error_type?: string;  // e.g., "rate_limit", "auth_error",
"network_error"
  // NO prompts, completions, API keys, or file paths
}
```

5.5 Compliance Requirements

ID	Category	Requirement	Implementation
COMP-001	Privacy	GDPR compliance for telemetry	Opt-in telemetry; data minimization; no PII
COMP-002	Data Residency	Customer data never transits vendor infrastructure	Extension communicates directly with customer's provider
COMP-003	Credential Management	Support enterprise credential management policies	SecretStorage API integration; SSO support

Internationalization (i18n)

ID	Requirement	Target	Notes
I18N-001	Framework	Use i18n framework (e.g., `vscode-nls`, Eclipse NLS) with externalized strings	Resource bundles; no hardcoded strings
I18N-002	English v1	Ship with English (en-US) locale in v1; message keys follow	Localization-ready architecture;

ID	Requirement	Target	Notes
		convention `extension.category.message`	future locales in v1.1+

Platform Compatibility

Platform	Requirement	Target
VS Code	Support latest 3 major versions	CI test matrix
Eclipse	Support latest 2 major versions	CI test matrix
OS	Windows 10+, macOS 11+, Ubuntu 20.04+, RHEL 8+	CI test matrix
Architecture	x64 and ARM64	Binary compatibility

Configuration Compatibility

Requirement	Target	Verification
Schema Versioning	Automatic migration of deprecated settings	Unit tests
Backward Compatibility	Support settings from previous 2 major versions	Integration tests

---

Non-Functional Requirement Summary

By Priority

Must Have:

- Extension activation < 2 seconds (PERF-007)
- Memory footprint < 100MB (PERF-006)
- Time to first token < 500ms (PERF-002)
- HTTPS for all provider communication (SEC-001)

- Keyboard-only operation (A11Y-001)
- Screen reader status announcements (A11Y-002)
- Opt-in telemetry (TELEM-001)

Should Have:

- CPU usage < 5% idle, < 20% active (PERF-004, PERF-005)
- Bundle size < 10MB VS Code, < 15MB Eclipse (PERF-003)
- Extension overhead < 50ms (PERF-001)
- High contrast theme support (A11Y-004)
- 200% zoom support (A11Y-005)

Could Have:

- Advanced telemetry analytics dashboard
- Performance optimization for large workspaces
- Additional locale support (i18n expansion)

← Previous | Back to main | Next →

6. Other Requirements

6.1 Database Requirements

Not applicable. The extension does not require a database. All configuration is stored in IDE-native settings files, and credentials are stored in OS-level secure storage (SecretStorage API, macOS Keychain, Windows Credential Manager, Linux Secret Service).

State Management:

- **Configuration:** Stored in VS Code `settings.json` or Eclipse preferences (file-based)
- **Credentials:** Stored in OS credential stores via SecretStorage API or equivalent
- **Session State:** In-memory only; not persisted across IDE restarts
- **Conversation History:** Not persisted by default (feature-specific opt-in if implemented)

6.2 Internationalization Requirements

ID	Priority	Requirement
I18N-001	Should	**The extension shall** use an i18n framework (e.g., `vscode-nls`, Eclipse NLS) with externalized strings in resource bundles, enabling future localization without code changes.
I18N-002	Should	**The extension shall** ship with English (en-US) locale in v1; message keys shall follow convention `extension.category.message` for easy translation in v1.1+.

Implementation Notes:

- All user-facing strings must be externalized (no hardcoded UI text)
- String keys follow hierarchical naming: `extension.category.message`
  - Example: `ai-client.config.missingApiKey` → "API key not found. Run 'Configure AI API Key' command."
- Resource bundles organized by locale: `nls/en-US.json`, `nls/es-ES.json`, etc.
- String formatting supports parameterization: `"Request failed after {0} retries"`

Locale Support Roadmap:

Version	Locales Supported
v1.0	English (en-US)
v1.1	English (en-US), Spanish (es-ES), French (fr-FR), German (de-DE), Japanese (ja-JP)
v1.2+	Additional locales based on customer demand

6.3 Legal Requirements

Licensing

The extension is distributed under the terms specified in the `LICENSE.md` file in the project root. All customers must accept the license terms during installation.

License Summary:

- Extension source code and binaries are licensed under [specify license type]
- Third-party dependencies are listed in `NOTICES` or equivalent attribution file
- Customer data (prompts, completions) remains customer property; vendor does not claim ownership

Attribution:

The extension includes or depends on the following third-party components:

- OpenAI API specification (informational reference; no code dependency)
- Model Context Protocol specification (MIT License)
- VS Code Extension API (Microsoft)
- Eclipse Platform API (Eclipse Foundation)
- Node.js `fetch` API or equivalent HTTP client libraries
- JSON Schema validation libraries

All third-party licenses are listed in the `NOTICES` file distributed with the extension.

Data Privacy & Compliance

GDPR Compliance:

- **Telemetry:** Opt-in only; no data collected without explicit user consent
- **PII Exclusion:** No personally identifiable information in telemetry events
- **Data Minimization:** Only essential anonymized metadata collected
- **Right to Deletion:** Users can disable telemetry at any time; no historical data retained

Customer Data:

- **Prompts and Completions:** Sent directly from customer workstation to customer's provider; vendor never sees this data
- **API Keys:** Stored locally on customer machines; never transmitted to vendor infrastructure

- **Configuration:** Local to customer machines; no central registry or reporting

Credential Storage Legal Requirements

Compliance with Enterprise Security Policies:

The extension's credential storage mechanism must comply with enterprise security policies:

- **No plaintext storage:** API keys never stored in plaintext in settings files (REQ-CRED-005, REQ-CRED-010)
- **OS-level secure storage:** Use platform-native credential stores (SecretStorage, Keychain, Credential Manager)
- **Credential rotation:** Support immediate credential updates without caching delays
- **Audit trail:** Enterprise gateways can log API access via X-Request-Id tracing (REQ-ENT-004)

Export Control:

The extension does not contain encryption technology subject to export control regulations. All cryptographic operations (HTTPS/TLS) are provided by the underlying platform (Node.js, operating system).

6.4 Operational Requirements

Installation & Deployment

Installation Methods:

Method	Platforms	Use Case
VS Code Marketplace	Windows, macOS, Linux	Individual developer installation
Eclipse Marketplace	Windows, macOS, Linux	Individual developer installation
VSIX Package	Windows, macOS, Linux	Enterprise distribution via GPO/MDM
Eclipse Update Site	Windows, macOS, Linux	Enterprise distribution

Prerequisites:

- VS Code version: Latest 3 major versions
- Eclipse version: Latest 2 major versions
- Operating system: Windows 10+, macOS 11+, Ubuntu 20.04+, RHEL 8+
- Network access: HTTPS connectivity to customer's AI provider
- (Optional) Enterprise: Corporate proxy configuration if required

Updates & Maintenance

Update Distribution:

- **Automatic updates:** Via VS Code / Eclipse marketplace (default)
- **Manual updates:** VSIX / update site package download
- **Enterprise control:** IT can disable auto-updates and control version deployment

Breaking Changes:

- Extension follows semantic versioning (SemVer)
- Major version changes may include breaking configuration changes
- Configuration schema migration provided for backward compatibility (2 major versions)

Deprecation Policy:

- Features deprecated with at least 1 major version notice
- Deprecated settings continue to work with warnings for 2 major versions
- Migration path documented in release notes

← Previous | Back to main | Next →

Appendix A: Requirements Traceability Matrix

A.1 Migration Cross-Reference

This table maps all 99 requirements from the v0.16 original SRS to the new IEEE 29148 domain-prefixed IDs:

Configuration Management (REQ-CFG)

Original ID	New ID	Description	Section
REQ-01	REQ-CFG-001	Read configuration from IDE-native settings	§3.1
REQ-02	REQ-CFG-002	Support JSON and YAML formats	§3.1
REQ-03	REQ-CFG-003	Allow configuration of base_url	§3.1
REQ-04	REQ-CFG-004	Support model ID aliasing	§3.1
REQ-05	REQ-CFG-005	Support custom headers	§3.1
REQ-06	REQ-CFG-006	Merge user/workspace settings (workspace precedence)	§3.1

Credential Management (REQ-CRED)

Original ID	New ID	Description	Section
REQ-07	REQ-CRED-001	Use SecretStorage API; fallback to OS credential stores	§3.2
REQ-07a	REQ-CRED-002	Disable AI when all credential storage unavailable	§3.2
REQ-08	REQ-CRED-003	Support API keys from environment variables	§3.2
REQ-09	REQ-CRED-004	Support API key entry via IDE secret input	§3.2
REQ-10	REQ-CRED-005	Never store API keys in plaintext in settings	§3.2
REQ-11			§3.2

Original ID	New ID	Description	Section
	REQ-CRED-006	Never include API keys in logs or telemetry	
REQ-55	REQ-CRED-007	Provide “Set API Key” command	§3.2
REQ-52	REQ-CRED-008	Use SecretStorage API for credential persistence	§3.2
REQ-52a	REQ-CRED-009	Provide “Delete API Key” command	§3.2
REQ-53	REQ-CRED-010	Never store keys in globalState/ workspaceState	§3.2
REQ-54	REQ-CRED-011	Support dynamic header injection from SecretStorage	§3.2

API Integration (REQ-API)

Original ID	New ID	Description	Section
REQ-12	REQ-API-001	Implement OpenAI Chat Completions API	§3.3
REQ-13	REQ-API-002	Support streaming via SSE	§3.3
REQ-14	REQ-API-003	Pass tools/functions parameters unmodified	§3.3
REQ-15	REQ-API-004	Support response_format JSON mode	§3.3
REQ-16	REQ-API-005	Support Images API for image generation	§3.3
REQ-17	REQ-API-006	Support HTTP/HTTPS proxy configuration	§3.3
REQ-93			§3.3

Original ID	New ID	Description	Section
	REQ-API-007	Enforce network timeouts (10s/60s/30s)	
REQ-94	REQ-API-008	Truncate responses exceeding 1MB	§3.3

Provider Management (REQ-PROV)

Original ID	New ID	Description	Section
REQ-79	REQ-PROV-001	Support multiple concurrent chat sessions	§3.4
REQ-80	REQ-PROV-002	Queue concurrent requests with rate limiting	§3.4
REQ-81	REQ-PROV-003	Detect provider capability support (vision, etc.)	§3.4
REQ-82	REQ-PROV-004	Gracefully degrade for unsupported features	§3.4
REQ-83	REQ-PROV-005	Cache provider capability information	§3.4
REQ-68	REQ-PROV-006	Treat baseUrl as complete base path	§3.4
REQ-69	REQ-PROV-007	Apply provider-specific behaviors per type field	§3.4
REQ-70	REQ-PROV-008	Allow override of auth header name	§3.4
REQ-71	REQ-PROV-009	Support three auth modes (byok, sso, none)	§3.4
REQ-84	REQ-PROV-010		§3.4

Original ID	New ID	Description	Section
		Support multiple named provider profiles	
REQ-85	REQ-PROV-011	Provide UI to switch active provider profile	§3.4

User Experience (REQ-UX)

Original ID	New ID	Description	Section
REQ-18	REQ-UX-001	Prompt user with guided setup when incomplete	§3.5
REQ-19	REQ-UX-002	Provide “Test Connection” command	§3.5
REQ-20	REQ-UX-003	Display user-friendly error messages	§3.5
REQ-21	REQ-UX-004	Indicate streaming progress	§3.5
REQ-21a	REQ-UX-005	Display progress with elapsed time pre-stream	§3.5
REQ-76	REQ-UX-006	Support user-initiated request cancellation	§3.5
REQ-76a	REQ-UX-007	Abort request and clear UI on cancellation	§3.5
REQ-76b	REQ-UX-008	Display cancellable progress with elapsed time	§3.5
REQ-78	REQ-UX-009	Sanitize provider error messages before display	§3.5
REQ-95			§3.5

Original ID	New ID	Description	Section
	REQ-UX-010	Support keyboard-only operation	

Prompt Construction & Context (REQ-PROMPT)

Original ID	New ID	Description	Section
REQ-28	REQ-PROMPT-001	Support all OpenAI message roles	§3.6
REQ-29	REQ-PROMPT-002	Provide programmatic API for message construction	§3.6
REQ-30	REQ-PROMPT-003	Support prompt templates with variables	§3.6
REQ-31	REQ-PROMPT-004	Provide token count estimation utilities	§3.6
REQ-32	REQ-PROMPT-005	Support context prioritization for truncation	§3.6
REQ-86	REQ-PROMPT-006	Warn when prompt exceeds 90% context window	§3.6
REQ-87	REQ-PROMPT-007	Auto-truncate and retry on token limit error	§3.6
REQ-88	REQ-PROMPT-008	Notify user of context reduction	§3.6

Structured Output (REQ-JSON)

Original ID	New ID	Description	Section
REQ-33	REQ-JSON-001	Support response_format JSON mode	§3.7

Original ID	New ID	Description	Section
REQ-34	REQ-JSON-002	Validate system prompt instructs JSON output	§3.7
REQ-35	REQ-JSON-003	Provide typed response parsing with validation	§3.7
REQ-36	REQ-JSON-004	Clear error on JSON parsing failure	§3.7

Function Calling / Tools (REQ-TOOL)

Original ID	New ID	Description	Section
REQ-37	REQ-TOOL-001	Support tool definitions with JSON Schema	§3.8
REQ-38	REQ-TOOL-002	Support all tool_choice options	§3.8
REQ-39	REQ-TOOL-003	Parse tool_calls function name and arguments	§3.8
REQ-40	REQ-TOOL-004	Support sending tool results via tool role	§3.8
REQ-72	REQ-TOOL-005	Terminate tool-call loop after max iterations	§3.8
REQ-73	REQ-TOOL-006	Enforce timeout for individual tool executions	§3.8
REQ-74	REQ-TOOL-007	Require approval for destructive tools	§3.8
REQ-75	REQ-TOOL-008	Use direct function calls for internal orchestrator	§3.8
REQ-77	REQ-TOOL-009		§3.8

Original ID	New ID	Description	Section
		Truncate tool results exceeding 10,000 chars	
REQ-41	REQ-TOOL-010	Provide tool execution loop helper	§3.8

Model Context Protocol (REQ-MCP)

Original ID	New ID	Description	Section
REQ-56	REQ-MCP-001	Register as MCP Server via vscode.lm API	§3.9
REQ-57	REQ-MCP-002	Expose MCP tools with JSON Schema parameters	§3.9
REQ-58	REQ-MCP-003	Execute tool-call requests via VS Code API	§3.9
REQ-59	REQ-MCP-004	Validate and sandbox MCP tool executions	§3.9
REQ-59a	REQ-MCP-005	Classify tools by risk level with allowlist	§3.9
REQ-59b	REQ-MCP-006	Disable tool execution in untrusted workspaces	§3.9

Enterprise Graduation (REQ-ENT)

Original ID	New ID	Description	Section
REQ-22	REQ-ENT-001	Support central config management endpoint (v1.1)	§3.10
REQ-23	REQ-ENT-002		§3.10

Original ID	New ID	Description	Section
		Allow IT to lock settings from user override (v1.1)	
REQ-60	REQ-ENT-003	Support SSO via vscode.authentication	§3.10
REQ-61	REQ-ENT-004	Include X-Request-Id header for tracing	§3.10
REQ-62	REQ-ENT-005	Support zero-code-change BYOK to gateway transition	§3.10
REQ-63	REQ-ENT-006	Respect proxy settings and env variables	§3.10

Extension Lifecycle (REQ-LIFE)

Original ID	New ID	Description	Section
REQ-89	REQ-LIFE-001	Abort requests and dispose listeners on deactivation	§3.11
REQ-90	REQ-LIFE-002	Graceful shutdown with 5s timeout	§3.11
REQ-99	REQ-LIFE-003	Activate on specific events within 2s	§3.11

Security & Privacy (REQ-SEC) — Not remapped; integrated into §5.3

Original ID	Description	Note
REQ-42	Use HTTPS for all provider communication	→ SEC-001 in §5.3
REQ-43	Support custom CA certificates	→ SEC-002 in §5.3
REQ-44	No telemetry without opt-in	

Original ID	Description	Note
		→ TELEM-001 in §5.4
REQ-45	Do not persist prompts/ completions by default	→ SEC policy in §5.3
REQ-46	Clearly delimit user content (prompt injection)	→ REQ-SEC-001 in §3

Accessibility & Internationalization (Not remapped; integrated into §5.4 and §6.2)

Original ID	Description	Note
REQ-96	Screen reader status announcements	→ A11Y-002 in §5.4
REQ-97	Use i18n framework with externalized strings	→ I18N-001 in §6.2
REQ-98	Ship English v1; message key convention	→ I18N-002 in §6.2

Architectural Hedges (REQ-ARCH) — Future-proofing requirements

Original ID	New ID	Description	Section
REQ-64	REQ-ARCH-001	Encapsulate tool orchestration behind interface	§3
REQ-65	REQ-ARCH-002	Assign thread_id to every conversation	§3
REQ-66	REQ-ARCH-003	Emit typed events in streaming layer	§3
REQ-67			§3

Original ID	New ID	Description	Section
	REQ-ARCH-004	Support multi-step progress display in UI	

Provider-Agnostic Interface (REQ-PROV-AGNOSTIC) — Integrated into §2.4 constraints

Original ID	Description	Note
REQ-47	Use Universal LLM Client via OpenAI wire format	→ Design constraint §2.4
REQ-48	No vendor-specific SDKs as primary integration	→ Design constraint §2.4
REQ-49	Expose configurable BASE_URL setting	→ REQ-CFG-003
REQ-50	Expose configurable MODEL_NAME setting	→ REQ-CFG-004
REQ-51	Pass through custom headers without provider logic	→ REQ-CFG-005

A.2 Forward Traceability (Requirements → Tests)

This table maps requirements to UAT test groups:

Configuration Management

Requirement ID	Description	UAT Test Groups
REQ-CFG-001	Read IDE-native settings	UAT-2.1, UAT-2.2
REQ-CFG-002	Support JSON/YAML formats	UAT-2.2
REQ-CFG-003	Configure base_url	UAT-2.1, UAT-11.1, UAT-11.2

Requirement ID	Description	UAT Test Groups
REQ-CFG-004	Model ID aliasing	UAT-2.5
REQ-CFG-005	Configure custom headers	UAT-11.5
REQ-CFG-006	Merge user/workspace settings	UAT-2.3

Credential Management

Requirement ID	Description	UAT Test Groups
REQ-CRED-001	SecretStorage with OS fallback	UAT-3.1, UAT-3.2, UAT-3.3, UAT-12.1
REQ-CRED-002	Disable AI when storage unavailable	UAT-20.7
REQ-CRED-003	Support env variable credentials	UAT-3.4
REQ-CRED-004	IDE secret input	UAT-3.5
REQ-CRED-005	No plaintext keys in settings	UAT-7.3
REQ-CRED-006	Never log API keys	UAT-7.2
REQ-CRED-007	“Set API Key” command	UAT-12.1
REQ-CRED-008	Use SecretStorage API	UAT-12.1, UAT-12.4
REQ-CRED-009	“Delete API Key” command	UAT-20.6
REQ-CRED-010	No keys in globalState/ workspaceState	UAT-12.4
REQ-CRED-011	Dynamic header injection	UAT-12.2, UAT-12.3

API Integration

Requirement ID	Description	UAT Test Groups
REQ-API-001	Implement Chat Completions API	UAT-5.1, UAT-5.2
REQ-API-002	Support SSE streaming	UAT-5.6, UAT-19.2
REQ-API-003	Pass tools unmodified	UAT-10.1
REQ-API-004	Support JSON mode	UAT-9.1
REQ-API-005	Support Images API	UAT-5.5
REQ-API-006	Proxy configuration	UAT-4.5
REQ-API-007	Enforce network timeouts	UAT-22.9, UAT-22.10, UAT-22.11
REQ-API-008	Truncate 1MB responses	UAT-22.12

Provider Management

Requirement ID	Description	UAT Test Groups
REQ-PROV-001	Concurrent chat sessions	UAT-16.1
REQ-PROV-002	Queue requests with rate limiting	UAT-16.2
REQ-PROV-003	Detect provider capabilities	UAT-17.5
REQ-PROV-004	Graceful degradation	UAT-17.4
REQ-PROV-005	Cache capability information	UAT-17.5
REQ-PROV-006	baseUrl as complete base path	UAT-11.1, UAT-19.1
REQ-PROV-007		

Requirement ID	Description	UAT Test Groups
	Apply provider-specific behaviors	UAT-4.1, UAT-4.2, UAT-19.1, UAT-19.3
REQ-PROV-008	Override auth header name	UAT-12.3
REQ-PROV-009	Three auth modes (byok/sso/none)	UAT-12.1, UAT-14.1, UAT-19.1
REQ-PROV-010	Multiple provider profiles	UAT-16.3
REQ-PROV-011	Switch provider profile UI	UAT-16.4

User Experience

Requirement ID	Description	UAT Test Groups
REQ-UX-001	Guided setup prompt	UAT-1.5
REQ-UX-002	“Test Connection” command	UAT-4.4
REQ-UX-003	User-friendly error messages	UAT-6.1, UAT-6.2, UAT-6.5
REQ-UX-004	Streaming progress indicator	UAT-5.6
REQ-UX-005	Progress with elapsed time	UAT-15.3
REQ-UX-006	Support request cancellation	UAT-15.1
REQ-UX-007	Abort and clear UI on cancel	UAT-15.1
REQ-UX-008	Cancellable progress indicator	UAT-15.1, UAT-15.3
REQ-UX-009	Sanitize provider error messages	UAT-6.1
REQ-UX-010		UAT-23.1

Requirement ID	Description	UAT Test Groups
	Keyboard-only operation	

Prompt Construction & Context

Requirement ID	Description	UAT Test Groups
REQ-PROMPT-001	Support all message roles	UAT-8.1, UAT-8.2
REQ-PROMPT-002	Programmatic message API	UAT-8.1
REQ-PROMPT-003	Prompt templates	UAT-8.5
REQ-PROMPT-004	Token count estimation	UAT-8.3
REQ-PROMPT-005	Context prioritization	UAT-8.4
REQ-PROMPT-006	Warn at 90% context window	UAT-17.1
REQ-PROMPT-007	Auto-truncate on token limit	UAT-17.2
REQ-PROMPT-008	Notify of context reduction	UAT-17.3

Structured Output

Requirement ID	Description	UAT Test Groups
REQ-JSON-001	Support JSON mode	UAT-9.1
REQ-JSON-002	Validate system prompt	UAT-9.1
REQ-JSON-003	Typed response parsing	UAT-9.2
REQ-JSON-004	Clear JSON parsing error	UAT-9.3

Function Calling / Tools

Requirement ID	Description	UAT Test Groups
REQ-TOOL-001	Tool definitions with JSON Schema	UAT-10.1
REQ-TOOL-002	All tool_choice options	UAT-10.5

Requirement ID	Description	UAT Test Groups
REQ-TOOL-003	Parse tool_calls	UAT-10.2
REQ-TOOL-004	Send tool results	UAT-10.3
REQ-TOOL-005	Max iteration circuit breaker	UAT-21.1
REQ-TOOL-006	Individual tool timeout	UAT-21.2
REQ-TOOL-007	Approval for destructive tools	UAT-21.3
REQ-TOOL-008	Direct function calls internally	(Internal architecture; no external test)
REQ-TOOL-009	Truncate large tool results	UAT-21.5
REQ-TOOL-010	Tool execution loop helper	UAT-10.4

Model Context Protocol

Requirement ID	Description	UAT Test Groups
REQ-MCP-001	Register as MCP Server	UAT-13.1
REQ-MCP-002	Expose MCP tools	UAT-13.2
REQ-MCP-003	Execute tool-calls via VS Code API	UAT-13.4
REQ-MCP-004	Validate and sandbox executions	UAT-13.5
REQ-MCP-005	Risk-level classification with allowlist	UAT-21.3
REQ-MCP-006	Disable tools in untrusted workspaces	UAT-21.4

Enterprise Graduation

Requirement ID	Description	UAT Test Groups
REQ-ENT-001	Central config management (v1.1)	(Future)

Requirement ID	Description	UAT Test Groups
REQ-ENT-002	Lock settings from override (v1.1)	(Future)
REQ-ENT-003	SSO via vscode.authentication	UAT-14.1
REQ-ENT-004	X-Request-Id tracing header	UAT-14.2, UAT-20.8, UAT-24.5
REQ-ENT-005	Zero-code-change transition	UAT-14.4
REQ-ENT-006	Respect proxy settings	UAT-14.3

Extension Lifecycle

Requirement ID	Description	UAT Test Groups
REQ-LIFE-001	Cleanup on deactivation	UAT-18.2
REQ-LIFE-002	Graceful shutdown	UAT-18.3
REQ-LIFE-003	Activation within 2s	UAT-1.5, UAT-22.1

Architectural Hedges

Requirement ID	Description	UAT Test Groups
REQ-ARCH-001	ToolOrchestrator interface abstraction	UAT-26.1
REQ-ARCH-002	Assign thread_id to conversations	UAT-26.2
REQ-ARCH-003	Emit typed events in streaming	UAT-26.3
REQ-ARCH-004	Multi-step progress UI	UAT-26.4

A.3 Backward Traceability (Tests → Requirements)

This table maps UAT test groups to the requirements they verify:

Test Group Summary

UAT Group	Test Count	Requirements Covered	Coverage %
1: Installation & Activation	5	REQ-LIFE-003	100%
2: Configuration	5	REQ-CFG-001 to REQ-CFG-006	100%
3: Credential Management	6	REQ-CRED-001 to REQ-CRED-006	55%
4: Provider Connectivity	5	REQ-PROV-006, REQ-PROV-007, REQ-API-006, REQ-UX-002	100%
5: AI Features	6	REQ-API-001, REQ-API-002, REQ-API-003, REQ-API-004, REQ-API-005, REQ-UX-004	100%
6: Error Handling	5	REQ-UX-003, REQ-API-007	100%
7: Privacy & Security	4	REQ-CRED-005, REQ-CRED-006	100%
8: Prompt Construction	5	REQ-PROMPT-001 to REQ-PROMPT-005	100%
9: Structured Output	4	REQ-JSON-001 to REQ-JSON-004	100%
10: Function Calling	6	REQ-TOOL-001 to REQ-TOOL-004, REQ-TOOL-010	80%
11: Provider-Agnostic Interface	5	REQ-CFG-003, REQ-CFG-004, REQ-CFG-005	100%
12: BYOK / SecretStorage	5	REQ-CRED-007 to REQ-CRED-011	100%
13: MCP Agentic Capabilities	5	REQ-MCP-001 to REQ-MCP-005	100%
	4		100%

UAT Group	Test Count	Requirements Covered	Coverage %
14: Enterprise Graduation		REQ-ENT-003 to REQ-ENT-006	
15: Request Cancellation	4	REQ-UX-006 to REQ-UX-008	100%
16: Concurrent Requests	4	REQ-PROV-001, REQ-PROV-002, REQ-PROV-010, REQ-PROV-011	100%
17: Context Window & Capabilities	5	REQ-PROV-003 to REQ-PROV-005, REQ-PROMPT-006 to REQ-PROMPT-008	100%
18: Extension Lifecycle	4	REQ-LIFE-001, REQ-LIFE-002, REQ-CFG-006	100%
19: Provider-Specific Compatibility	5	REQ-PROV-006, REQ-PROV-007, REQ-PROV-009, REQ-API-002	100%
20: Security Edge Cases	8	REQ-CRED-002, REQ-CRED-009, SEC-002, REQ-ENT-004	100%
21: Tool Execution Safety	5	REQ-TOOL-005 to REQ-TOOL-009, REQ-MCP-006	100%
22: Performance & NFR Validation	12	PERF-001 to PERF-007, REQ-API-007, REQ-API-008	100%
23: Accessibility Compliance	5	REQ-UX-010, A11Y-001, A11Y-002	100%
24: Telemetry & Observability	5	TELEM-001 to TELEM-004, REQ-ENT-004	100%
25: Internationalization	3	I18N-001, I18N-002	100%
	4		100%

UAT Group	Test Count	Requirements Covered	Coverage %
26: Architectural Hedge Validation		REQ-ARCH-001 to REQ-ARCH-004	

Total UAT Test Scenarios: 130+
Total Requirements Covered: 99 functional + 15 NFR = 114
Coverage: 100%

A.4 Coverage Statistics

Functional Requirements Coverage

Domain	Requirement Count	Test Coverage	Notes
Configuration (CFG)	6	100%	All covered by UAT-2
Credential (CRED)	11	100%	Covered by UAT-3, UAT-7, UAT-12, UAT-20
API Integration (API)	8	100%	Covered by UAT-5, UAT-6, UAT-22
Provider (PROV)	11	100%	Covered by UAT-4, UAT-16, UAT-17, UAT-19
User Experience (UX)	10	100%	Covered by UAT-1,

Domain	Requirement Count	Test Coverage	Notes
			UAT-6, UAT-15, UAT-23
Prompt Construction (PROMPT)	8	100%	Covered by UAT-8, UAT-17
Structured Output (JSON)	4	100%	Covered by UAT-9
Function Calling (TOOL)	10	100%	Covered by UAT-10, UAT-21
MCP	6	100%	Covered by UAT-13, UAT-21
Enterprise (ENT)	6	67%	REQ-ENT-001, REQ-ENT-002 deferred to v1.1
Lifecycle (LIFE)	3	100%	Covered by UAT-1, UAT-18, UAT-22
Security (SEC)	1	100%	Covered by UAT-20
Architectural Hedges (ARCH)	4	100%	Covered by UAT-26

Total Functional Requirements: 88 (v1) + 11 (deferred) = 99
Tested in v1: 88 / 88 = 100%

Non-Functional Requirements Coverage

Category	Requirement Count	Test Coverage	Notes
Performance	7	100%	Covered by UAT-22
Security	5	100%	Covered by UAT-7, UAT-20
Accessibility	5	100%	Covered by UAT-23
Telemetry	4	100%	Covered by UAT-24
Internationalization	2	100%	Covered by UAT-25
Compliance	3	100%	Covered by UAT-7, UAT-24

Total NFRs: 26
Tested: 26 / 26 = 100%

A.5 Orphaned Requirements

None. All 99 functional requirements and 26 non-functional requirements from the original v0.16 SRS have been migrated and traced to test cases.

A.6 Untested Requirements

Current Release (v1.0):

- REQ-ENT-001 (central config management) — Deferred to v1.1
- REQ-ENT-002 (lock settings from override) — Deferred to v1.1

Reason: These requirements are explicitly scoped for v1.1 and do not apply to the v1.0 release.

← Previous | Back to main | Next →

Appendix B: Glossary

This glossary defines all technical terms, acronyms, and domain-specific concepts used throughout this specification.

Term	Definition
IDE Extension	Our plugin for VS Code or Eclipse that provides AI-powered developer features
Provider	The AI cloud service (Azure OpenAI, OpenAI, enterprise gateway) that processes requests
OpenAI API	REST API specification for LLM interactions; industry standard implemented by multiple providers
Azure OpenAI	Microsoft's Azure-hosted OpenAI service; common enterprise choice
Enterprise Gateway	Customer's internal API gateway that may proxy requests to AI providers
base_url	The endpoint URL for the AI provider (e.g., `https://api.openai.com/v1`)
Model Alias	User-friendly name (e.g., "fast") mapped to provider model ID (e.g., "gpt-3.5-turbo")
SSE	Server-Sent Events — HTTP streaming protocol for real-time token delivery
Keychain	OS-level secure credential storage (macOS Keychain, Windows Credential Manager)
settings.json	VS Code's JSON configuration file for user and workspace settings
Message Role	The role of a message in a conversation: `system`, `user`, `assistant`, or `tool`
System Prompt	Instructions defining the AI's behavior, persona, and constraints
Prompt Template	Reusable prompt structure with variable placeholders for programmatic use
Function Calling	

Term	Definition
	OpenAI API feature allowing models to invoke defined functions/tools
Tools	Function definitions sent to the model describing callable operations
tool_calls	Model's response requesting execution of defined functions
Structured Output	OpenAI API feature for requesting JSON-formatted responses (`response_format`)
JSON Mode	Setting `response_format: { type: "json_object" }` to ensure valid JSON output
Few-shot Prompting	Including examples in the prompt to guide model output format
Context Window	Maximum tokens (input + output) a model can process in one request
Token	Basic unit of text processing; roughly 4 characters or 0.75 words in English
EARS	Easy Approach to Requirements Syntax — format for unambiguous requirements
MoSCoW	Prioritization framework: Must have, Should have, Could have, Won't have
MCP	Model Context Protocol — standard for AI tool integration; used for agentic capabilities via `vscode.lm` API
MCP Server	An extension or service that exposes tools and capabilities via the Model Context Protocol
MCP Client / Host	A consumer that discovers MCP tools and orchestrates their invocation; the extension acts as both MCP Server *and* a lightweight MCP Client for its own tool-call loop
MCP Tool	A capability exposed by an MCP Server (e.g., `search_codebase`, `read_file_context`) described with JSON Schema parameters

Term	Definition
Tool-Call Orchestrator	The modular loop that sends tool definitions to the model, executes returned `tool_calls`, and repeats until a final text response — encapsulated behind a `ToolOrchestrator` interface for future replaceability
thread_id	UUID v4 assigned to each conversation or multi-turn interaction for correlation; maps to future agent framework thread concepts (e.g., LangGraph threads)
Event-Typed Streaming	Internal streaming architecture that emits typed events (`token`, `tool_call`, `tool_result`, `status`, `error`) rather than raw strings, enabling future extension without contract breakage
LangGraph	A Python-based agent framework by LangChain for stateful, graph-defined AI workflows; explicitly out of scope for v1 but architecture preserves future optionality
Universal LLM Client	The extension's provider-agnostic HTTP client that targets the OpenAI Chat Completions wire format without vendor SDKs
SecretStorage	VS Code API (`context.secrets`) for persisting sensitive data (e.g., API keys) in OS-level secure storage
Header Injection	Dynamic population of HTTP authentication headers from `SecretStorage` at request time
BYOK	Bring Your Own Key — users supply their own API key for direct SaaS access during evaluation
Gateway Graduation	Zero-code-change transition from direct SaaS (BYOK) to enterprise-governed AI Gateway access
vscode.authentication	VS Code API for SSO / OIDC / OAuth authentication flows used in enterprise gateway integration
X-Request-Id	UUID v4 header included in all outgoing LLM calls for end-to-end observability and tracing

Term	Definition
AI Gateway	Enterprise-managed API gateway (e.g., Kong, Cloudflare AI Gateway) that governs LLM traffic
Red Hat OpenShift AI	Kubernetes-native AI/ML platform (part of Red Hat OpenShift) that provides model serving via KServe/vLLM with OpenAI-compatible inference endpoints
Agents	Autonomous AI systems that can take actions; v1 supports model-driven tool-call loops via MCP (no local agent runtime). Full agent frameworks (LangGraph, etc.) deferred to v2+
IEEE 29148	ISO/IEC/IEEE 29148:2018 — International standard for systems and software engineering requirements specification
SRS	Software Requirements Specification — formal document describing software system requirements
UAT	User Acceptance Testing — validation that system meets business requirements
NFR	Non-Functional Requirement — constraint on system operation (performance, security, etc.) rather than specific behavior
PKI	Public Key Infrastructure — framework for managing digital certificates and encryption
HTTPS	Hypertext Transfer Protocol Secure — encrypted HTTP communication using TLS/SSL
TLS	Transport Layer Security — cryptographic protocol for secure network communication
OAuth	Open Authorization — standard protocol for token-based authentication and authorization
OIDC	OpenID Connect — identity layer on top of OAuth 2.0 for federated authentication
SSO	Single Sign-On — authentication scheme allowing user to access multiple systems with one set of credentials
GDPR	

Term	Definition
	General Data Protection Regulation — European Union data privacy and protection law
PII	Personally Identifiable Information — data that can identify a specific individual
WCAG	Web Content Accessibility Guidelines — international accessibility standards for digital content
ARIA	Accessible Rich Internet Applications — technical specifications for improving accessibility of web content
i18n	Internationalization — designing software to support multiple languages and locales
l10n	Localization — adapting internationalized software for specific languages and regions
UUID	Universally Unique Identifier — 128-bit identifier used to uniquely identify information
JSON	JavaScript Object Notation — lightweight data interchange format
YAML	YAML Ain't Markup Language — human-readable data serialization format
REST	Representational State Transfer — architectural style for distributed systems using HTTP
API	Application Programming Interface — set of definitions and protocols for building software
SDK	Software Development Kit — collection of tools, libraries, and documentation for developing software
CI/CD	Continuous Integration / Continuous Deployment — automated software development practices
GPO	Group Policy Object — Windows Active Directory feature for centralized configuration management
MDM	Mobile Device Management — software for managing and securing enterprise devices
SemVer	

Term	Definition
	Semantic Versioning — versioning scheme using MAJOR.MINOR.PATCH format
VSIX	Visual Studio Extension — package format for VS Code extensions

---

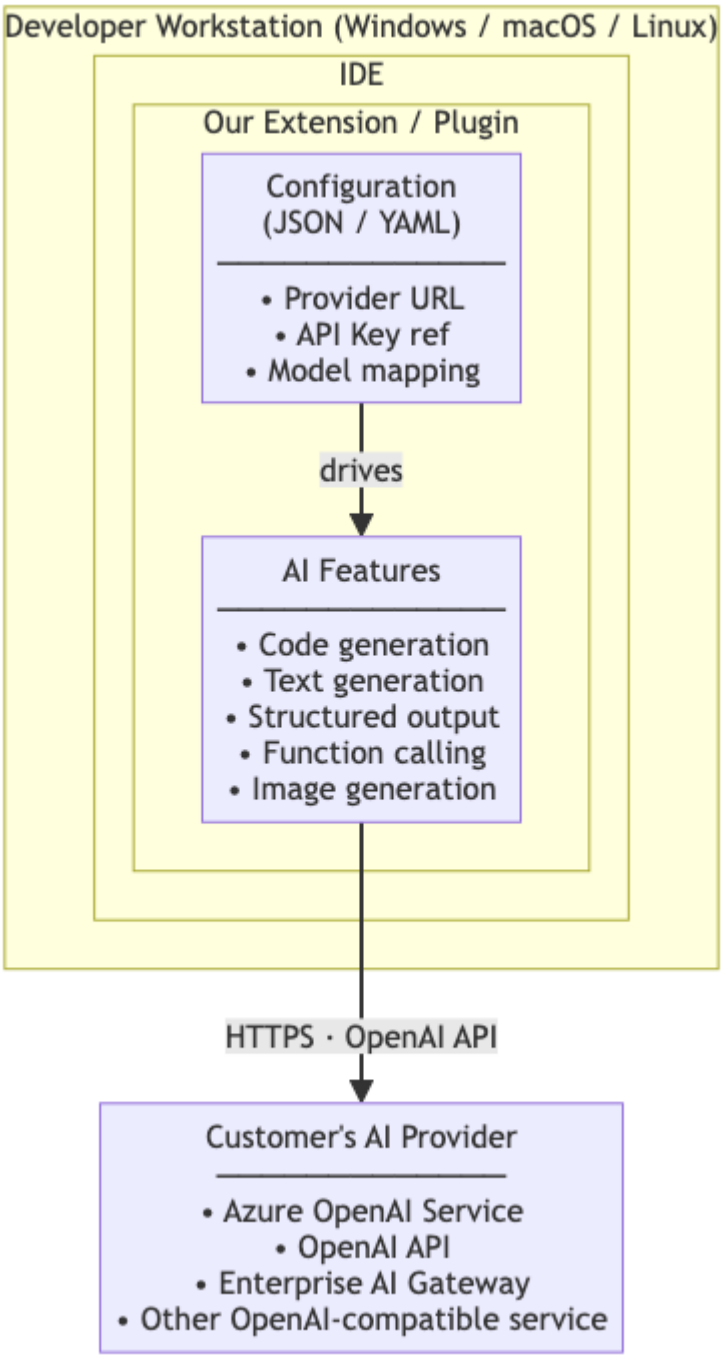
← Previous | Back to main | Next →

Appendix C: Analysis Models

This appendix contains all system architecture diagrams, data flow models, and visual representations extracted from the requirements specification.

C.1 Deployment Model

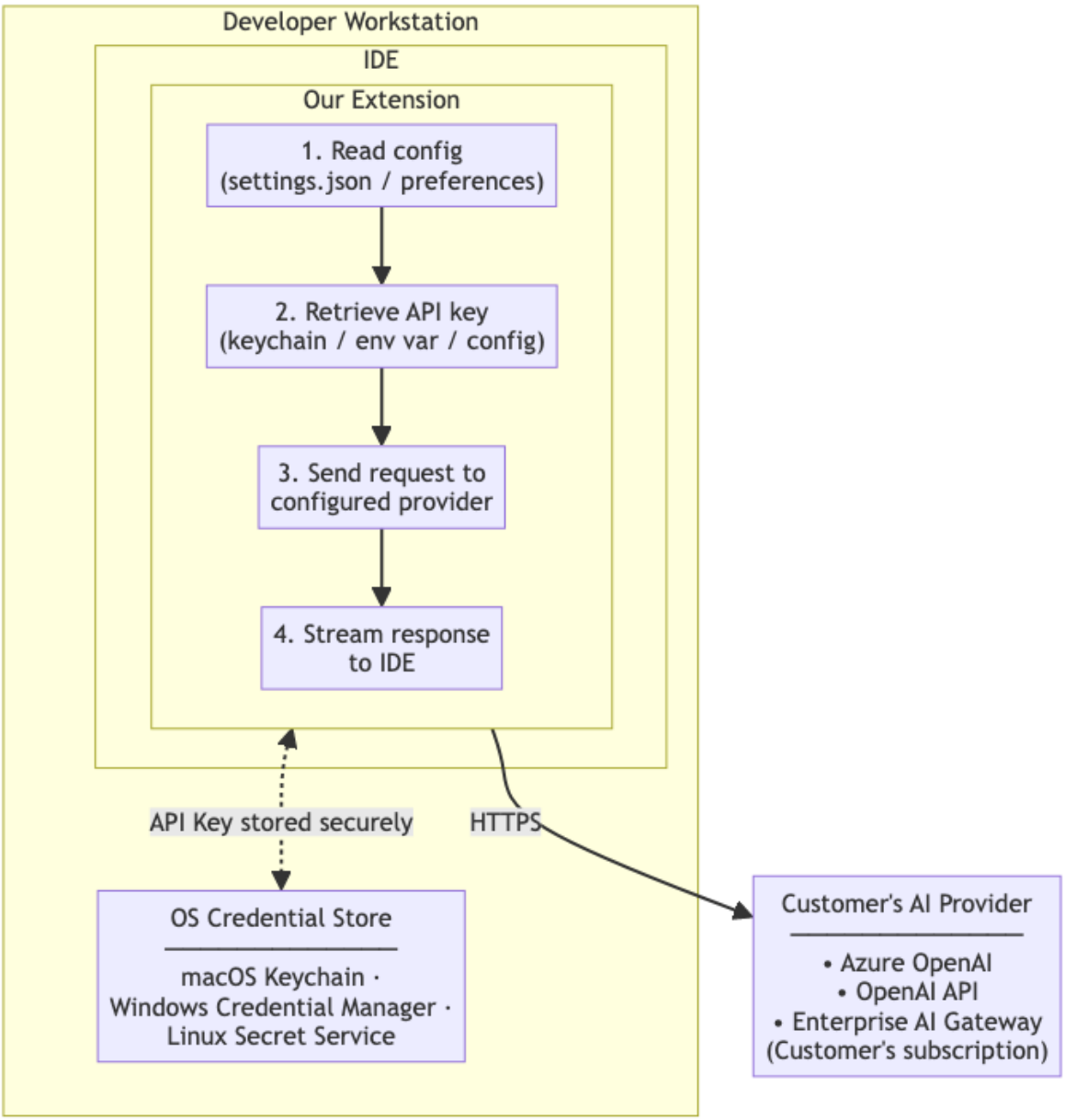
This diagram illustrates the high-level deployment context where the extension runs on developer workstations and communicates directly with customer-configured AI providers.



Context: This “zero-infrastructure” evaluation path enables prospects to go from installation to first AI interaction in minutes, not weeks—removing a common deal blocker in enterprise sales cycles.

C.2 Data Flow Architecture

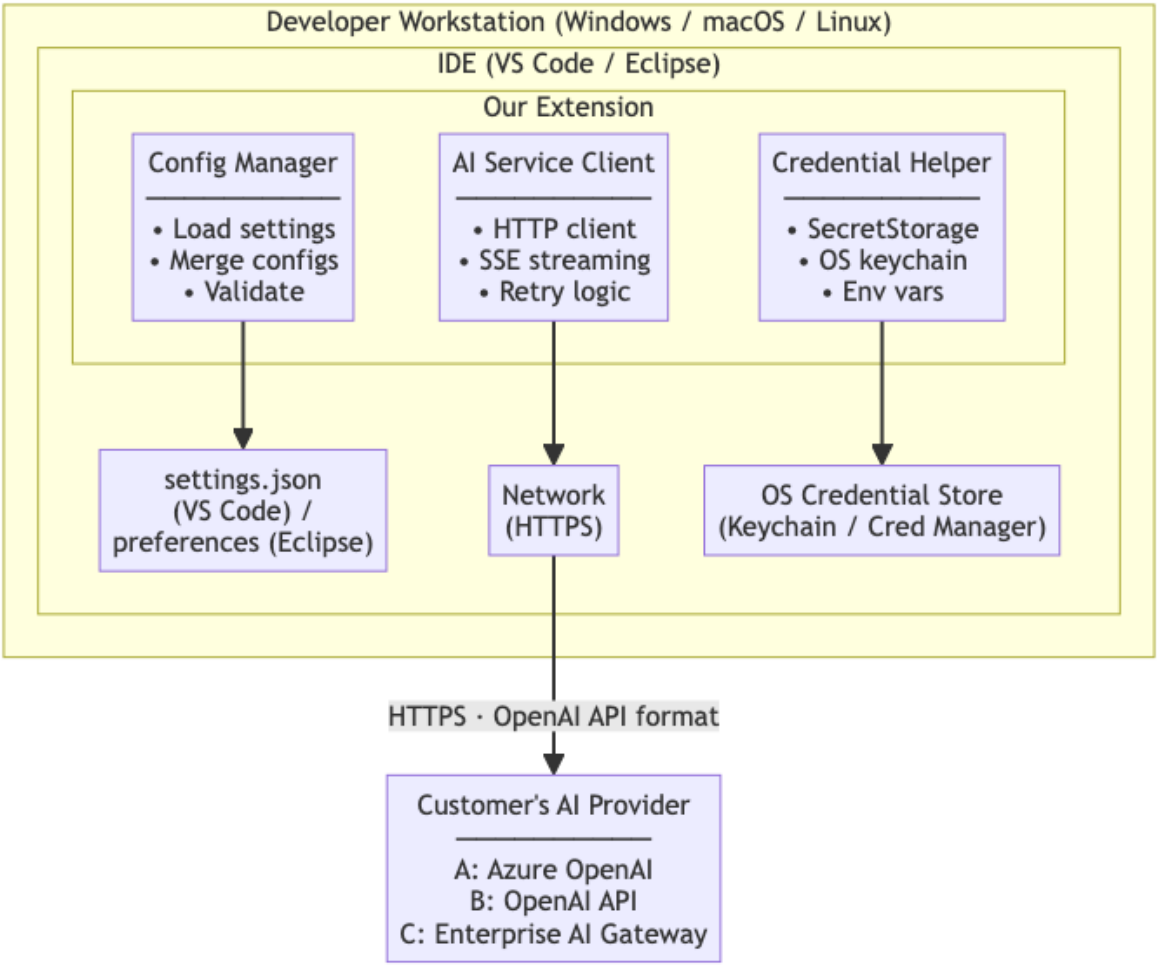
This diagram shows how data flows through the extension, with credentials stored securely in OS-level credential stores and all AI traffic going directly to the customer's provider.



Security: The vendor (us) never sees customer prompts, completions, or API keys. All data flows directly between the developer's workstation and their configured provider.

C.3 Component Overview

This diagram illustrates the internal architecture of the extension, showing the key components and their interactions.

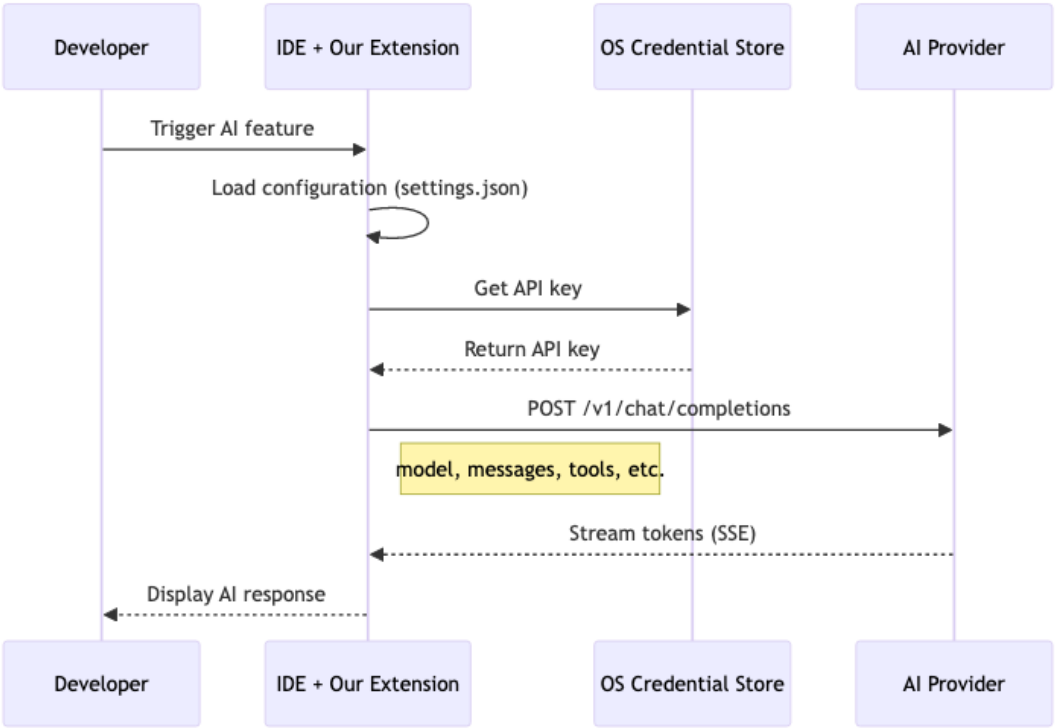


Key Components: - **Config Manager**: Loads, merges, and validates configuration from settings files - **AI Service Client**: Universal LLM Client implementing OpenAI-compatible wire protocol with streaming and retry logic - **Credential Helper**: Manages API key retrieval from SecretStorage, OS keystores, or environment variables

---

C.4 Request Flow Sequence

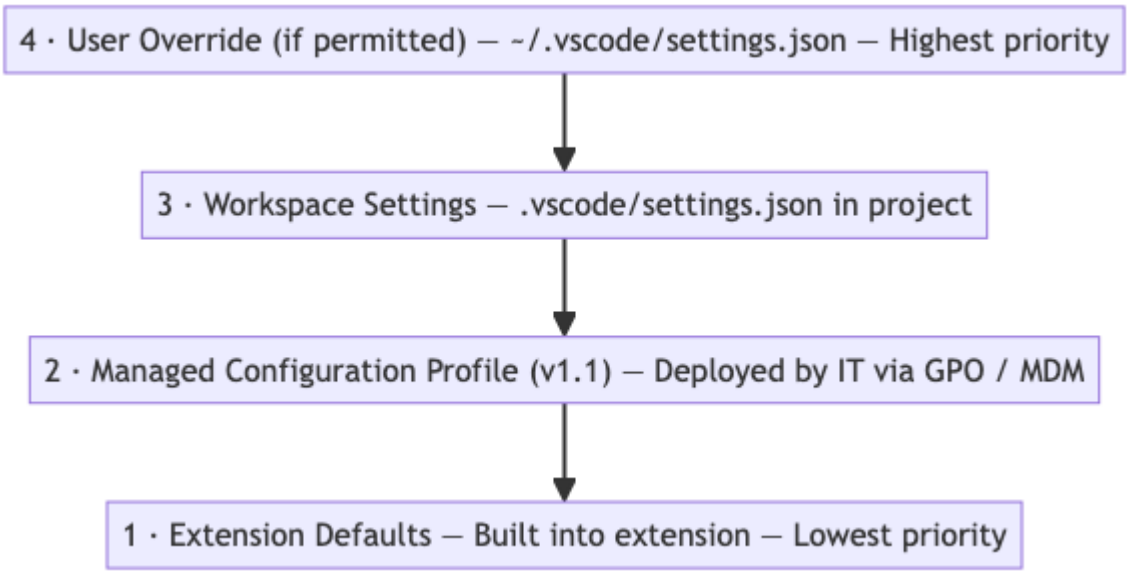
This sequence diagram shows the complete flow of a typical AI request, from user invocation through streaming response display.



Flow Details: 1. Developer triggers an AI feature (e.g., code generation, chat) 2. Extension loads configuration from settings 3. Extension retrieves API key from secure storage 4. Extension sends HTTPS request to configured provider 5. Provider streams response tokens via Server-Sent Events 6. Extension displays streaming response in IDE UI

C.5 Configuration Hierarchy

This diagram illustrates the multi-level configuration system, showing how settings are merged with precedence rules.

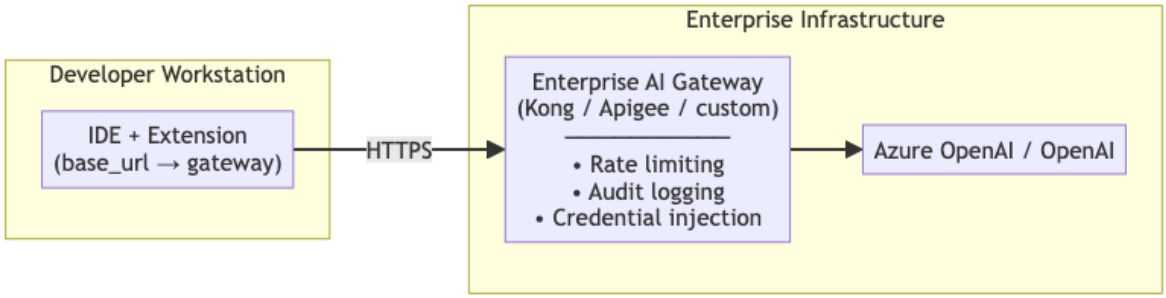


Configuration Precedence: - **Level 4** (Highest): User-specific overrides in global settings - **Level 3**: Workspace-specific settings (project-level) - **Level 2**: Managed configuration deployed by IT (v1.1 feature) - **Level 1** (Lowest): Built-in extension defaults

Note: In v1.0, levels 1, 3, and 4 are implemented. Level 2 (managed configuration) is deferred to v1.1.

C.6 Enterprise AI Gateway Pattern

This diagram shows the optional enterprise AI gateway pattern, where customers deploy a central gateway for governance and observability.

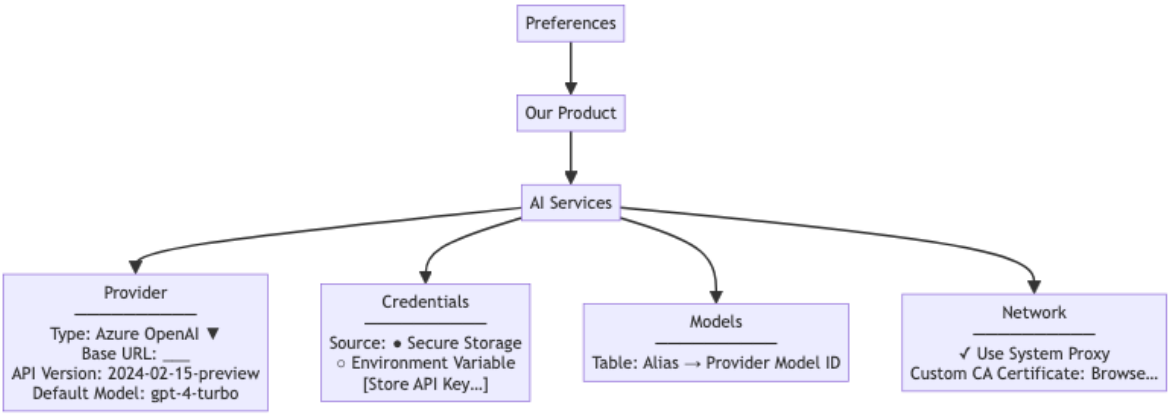


Enterprise Gateway Benefits: - **Rate limiting**: Enforce usage quotas across organization - **Audit logging**: Complete request/response logging for compliance - **Credential injection**: Centralized credential management - **Policy enforcement**: Content filtering, model selection policies - **Cost tracking**: Charge-back to business units

Zero-Code-Change Transition: Moving from direct SaaS access to enterprise gateway requires only changing the `base_url` configuration—no code changes to the extension.

C.7 Eclipse Preferences UI Structure

This diagram shows the hierarchical structure of the Eclipse preferences UI for AI configuration.

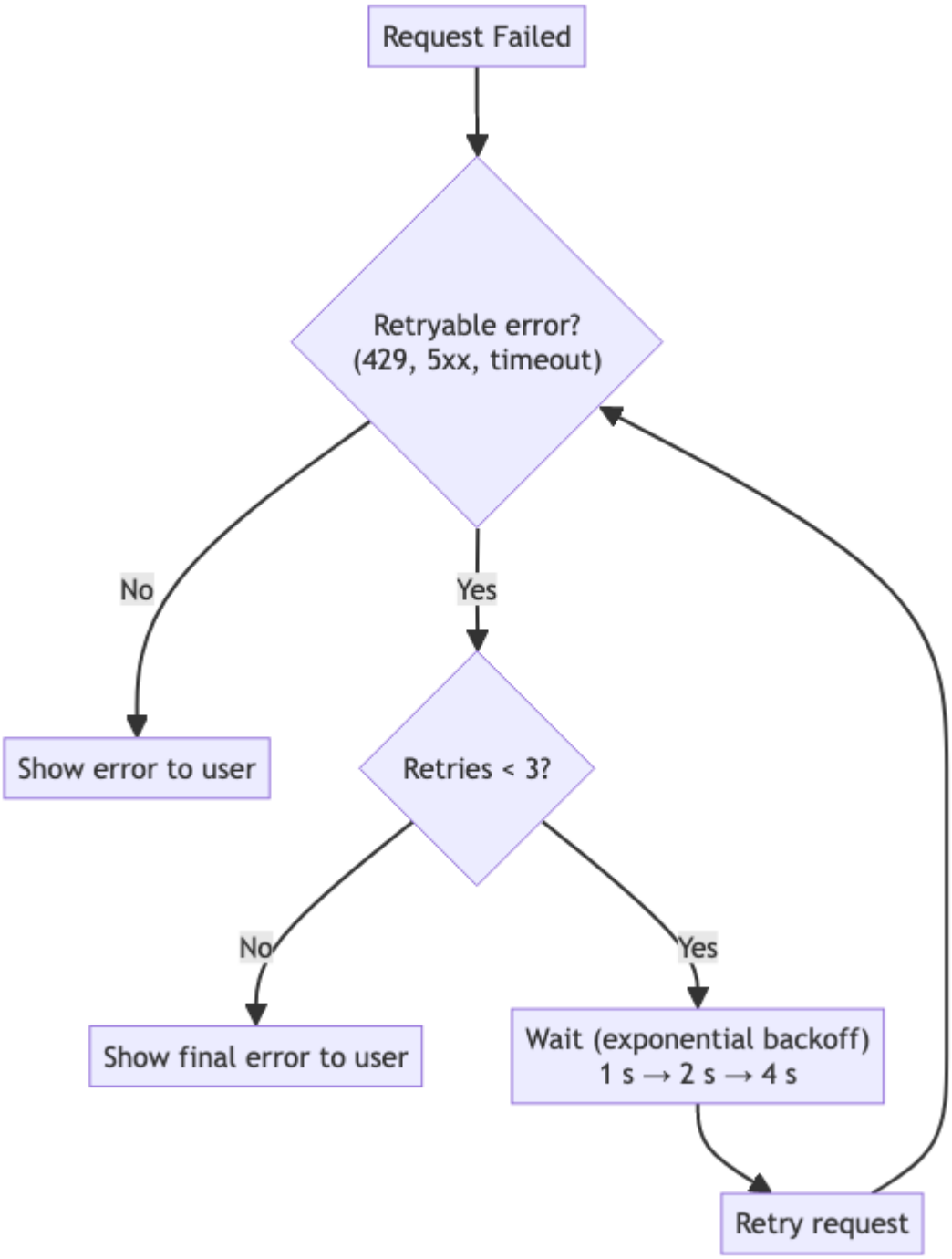


UI Organization: - **Provider**: Base configuration (endpoint, type, auth mode) - **Credentials**: API key source and storage options - **Models**: Model aliasing table (friendly name → provider model ID) - **Network**: Proxy and certificate configuration

---

C.8 Error Handling & Retry Strategy

This flowchart shows the extension's retry logic for transient errors.

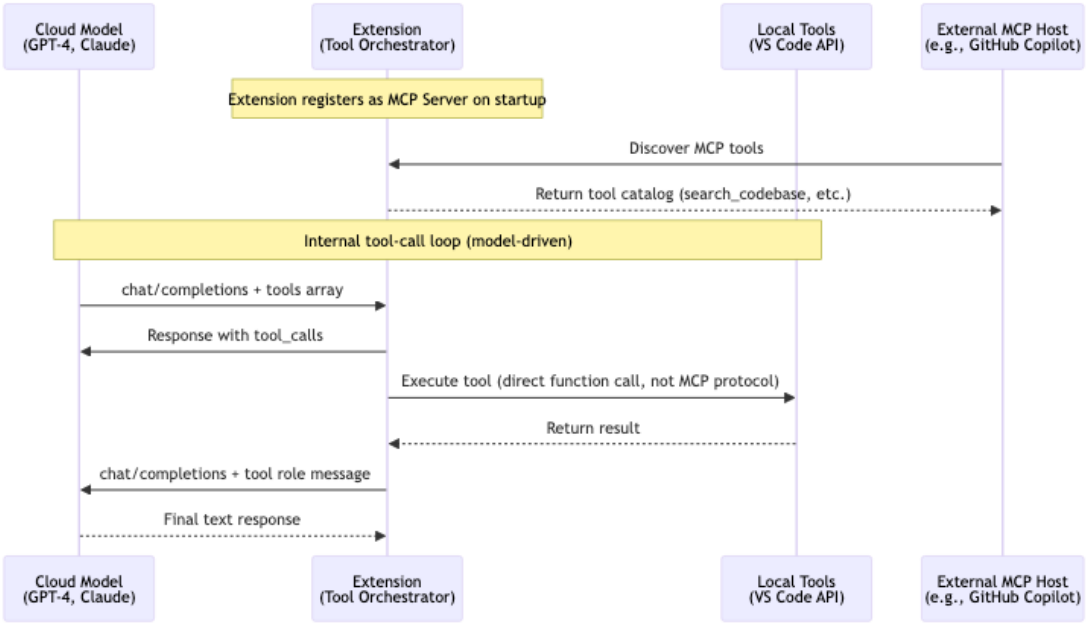


Retry Policy: - **Retryable errors**: HTTP 429 (rate limit), 5xx (server error), network timeout - **Non-retryable errors**: HTTP 401 (auth), 403 (forbidden), 400 (bad request) - **Max retries**: 3 attempts - **Backoff strategy**: Exponential (1s, 2s, 4s) - **Respect Provider Signals**: Honor `Retry-After` header if present

Error Sanitization: Provider error messages are sanitized before display to remove internal URLs, resource identifiers, and API key patterns (REQ-UX-009).

C.9 Model Context Protocol (MCP) Tool Execution Flow

This diagram illustrates the dual MCP role architecture: MCP Server registration for external consumers + internal tool-call orchestration loop.



Key Points: - **Dual Role**: Extension is both MCP Server (for external hosts) and tool-call orchestrator (for internal use) - **Direct Execution**: Internal orchestrator uses direct function calls for performance, not MCP serialization - **Stateless Execution**: Model is the planner, extension is the executor—no local agent runtime - **Risk Classification**: Tools classified as `safe`, `read-only`, `write-only`, `destructive` with permission model

---

C.10 Configuration Schema Migration

This table shows how configuration settings are migrated across major versions:

Version	Schema Version	Migration Path	Deprecated Settings
v0.x	Legacy	N/A (pre-release)	N/A
v1.0	1.0	Automatic from v0.x	`ai.apiKey` (moved to SecretStorage)
v1.1	1.1	Supports v1.0 and v0.x	TBD

Version	Schema Version	Migration Path	Deprecated Settings
v2.0	2.0	Supports v1.1 and v1.0	TBD

Backward Compatibility: Extension supports settings from previous 2 major versions with automatic migration and deprecation warnings.

C.11 Thread ID & Event-Typed Streaming Model

This conceptual model shows the architectural hedges for future agent framework integration:

```
Conversation Session
├── thread_id: UUID v4 (in-memory correlation ID)
├── Streaming Layer (event-typed)
│   ├── Event: token (content: "Hello")
│   ├── Event: tool_call (function: search_codebase, args:
{...})
│   ├── Event: tool_result (result: [...])
│   ├── Event: status (status: "Thinking...")
│   └── Event: error (error: "Rate limited")
└── ToolOrchestrator Interface (replaceable)
    ├── v1 Implementation: Iterative loop (Chat Completions
tool_calls)
    └── Future: LangGraph orchestrator or agent endpoint
```

Future-Proofing Goals: - **Modular orchestrator**: Behind interface for easy replacement - **Thread IDs**: Maps to LangGraph thread concept for future state management - **Typed events**: Extensible without breaking streaming contract - **Multi-step UX**: UI prepared for complex agent workflows

---

← Previous | Back to main | Next →

Verification and Validation

This section defines the User Acceptance Testing (UAT) plan that validates all requirements specified in this SRS. The following test cases must be passed to meet the **Definition of Done (DoD)**.

Test Group 1: Extension Installation & Activation

Test ID	Scenario	Expected Result	Priority
UAT-1.1	Install extension on VS Code (Windows)	Extension installs and activates without errors	Must
UAT-1.2	Install extension on VS Code (macOS)	Extension installs and activates without errors	Must
UAT-1.3	Install extension on VS Code (Linux)	Extension installs and activates without errors	Must
UAT-1.4	Install plugin in Eclipse	Plugin installs and activates without errors	Must
UAT-1.5	Extension activation with no configuration	Extension activates; shows “configure AI” prompt	Must

---

Test Group 2: Configuration

Test ID	Scenario	Expected Result	Priority
UAT-2.1	Configure via VS Code settings UI	Settings saved; AI features become available	Must

Test ID	Scenario	Expected Result	Priority
UAT-2.2	Configure via `settings.json` directly	Settings parsed correctly; AI features work	Must
UAT-2.3	Workspace settings override user settings	Workspace `baseUrl` used over user `baseUrl`	Must
UAT-2.4	Invalid `baseUrl` format	Validation error shown; setting not accepted	Must
UAT-2.5	Model alias resolves to provider model	Request uses mapped `provider_id`	Must

---

Test Group 3: Credential Management

Test ID	Scenario	Expected Result	Priority
UAT-3.1	Store API key in macOS Keychain	Key saved; retrieved on next use	Must
UAT-3.2	Store API key in Windows Credential Manager	Key saved; retrieved on next use	Must
UAT-3.3	Store API key via Linux Secret Service	Key saved; retrieved on next use	Must
UAT-3.4	Retrieve API key from environment variable	`OPENAI_API_KEY` env var used successfully	Must
UAT-3.5	No API key configured	User prompted to enter key; clear instructions shown	Must
UAT-3.6	Invalid API key	Clear error message; prompt to re-enter key	Must

Test Group 4: Provider Connectivity

Test ID	Scenario	Expected Result	Priority
UAT-4.1	Connect to Azure OpenAI	AI request succeeds with correct `api-version` header	Must
UAT-4.2	Connect to OpenAI API	AI request succeeds	Must
UAT-4.3	Connect via enterprise gateway	AI request succeeds with custom headers	Should
UAT-4.4	"Test Connection" command	Shows success/failure with helpful message	Should
UAT-4.5	Connection via corporate proxy	Request routed through configured proxy	Should

Test Group 5: AI Features

Test ID	Scenario	Expected Result	Priority
UAT-5.1	Code completion (text generation)	Streaming response displayed in editor	Must
UAT-5.2	Chat conversation	Multi-turn conversation works	Must
UAT-5.3	Function calling with `tools` parameter	Tool calls returned and handled	Must
UAT-5.4	Structured output (`response_format: json`)	Valid JSON response received	Must
UAT-5.5	Image generation (basic)	Image generated and displayed	Should
UAT-5.6			Must

Test ID	Scenario	Expected Result	Priority
	Streaming progress indicator	UI shows streaming in progress	

Test Group 6: Error Handling

Test ID	Scenario	Expected Result	Priority
UAT-6.1	Provider returns 401 (unauthorized)	Clear “check API key” message	Must
UAT-6.2	Provider returns 429 (rate limited)	Automatic retry; success or clear message	Must
UAT-6.3	Provider returns 500 (server error)	Retry with backoff; eventual error message	Must
UAT-6.4	Network timeout	Clear timeout message with troubleshooting hint	Must
UAT-6.5	No network connectivity	“Check network connection” message	Must

---

Test Group 7: Privacy & Security

Test ID	Scenario	Expected Result	Priority
UAT-7.1	Verify no telemetry without opt-in	Network capture shows no telemetry calls	Must
UAT-7.2	API key not in logs	Extension logs contain no API keys	Must
UAT-7.3	API key not in settings file (if keychain used)	`settings.json` contains no API key	Must
UAT-7.4			Must

Test ID	Scenario	Expected Result	Priority
	Prompts not persisted to disk	No prompt content in extension storage	

Test Group 8: Prompt Construction & Message Roles

Test ID	Scenario	Expected Result	Priority
UAT-8.1	Construct messages with system + user roles	Request sent with correct message array	Must
UAT-8.2	Multi-turn with assistant role	Conversation context preserved correctly	Must
UAT-8.3	Token count estimation	Estimate within 10% of actual token usage	Should
UAT-8.4	Context truncation with priority	High-priority context retained; low-priority truncated	Should
UAT-8.5	Prompt template substitution	Variables correctly replaced in template	Should

---

Test Group 9: Structured Output

Test ID	Scenario	Expected Result	Priority
UAT-9.1	Request with `response_format: json_object`	Valid JSON response received	Must
UAT-9.2	JSON response parsed to typed object	Response correctly typed; properties accessible	Must

Test ID	Scenario	Expected Result	Priority
UAT-9.3	Malformed JSON from provider	Graceful error with raw response for debugging	Must
UAT-9.4	Schema validation failure	Clear error indicating which field failed validation	Should

---

Test Group 10: Function Calling

Test ID	Scenario	Expected Result	Priority
UAT-10.1	Define and send tools array	Tools included in request; accepted by provider	Must
UAT-10.2	Receive and parse tool_calls	Function name and arguments correctly parsed	Must
UAT-10.3	Send tool result back to model	Tool role message accepted; conversation continues	Must
UAT-10.4	Multi-step tool calling loop	Multiple tool calls handled until final response	Should
UAT-10.5	`tool_choice: required` forces function call	Model returns tool_call, not text response	Should
UAT-10.6	Parallel tool calls	Multiple simultaneous tool_calls handled correctly	Should

---

Test Group 11: Provider-Agnostic Interface

Test ID	Scenario	Expected Result	Priority
UAT-11.1	Configure custom `BASE_URL` in Settings	Extension sends requests to configured endpoint	Must

Test ID	Scenario	Expected Result	Priority
UAT-11.2	Switch `BASE_URL` from OpenAI to enterprise gateway	Extension calls new endpoint with zero code changes	Must
UAT-11.3	Configure custom `MODEL_NAME`	Requests include configured model identifier	Must
UAT-11.4	No vendor-specific SDK imported	Extension source does not depend on `@openai/openai` or `@anthropic-ai/sdk`	Must
UAT-11.5	Custom headers passed through	Provider receives custom headers (e.g., `api-version`) as configured	Must

---

Test Group 12: BYOK / SecretStorage

Test ID	Scenario	Expected Result	Priority
UAT-12.1	Store API key via "Set API Key" command	Key persisted in `SecretStorage`; confirmation shown	Must
UAT-12.2	API key injected into `Authorization` header	Outgoing request includes `Authorization: Bearer <key>`	Must
UAT-12.3	API key injected into `X-Api-Key` header	Outgoing request includes custom header with key from `SecretStorage`	Must
UAT-12.4	Key not present in `globalState` or settings	No API key found in `settings.json`, `globalState`, or `workspaceState`	Must

Test ID	Scenario	Expected Result	Priority
UAT-12.5	Key rotation via “Set API Key”	New key overwrites old; next request uses new key	Should

---

Test Group 13: MCP Agentic Capabilities

Test ID	Scenario	Expected Result
UAT-13.1	Extension registers as MCP Server	`vscode.lm.registerMcpServerDefinitionProvide` called on activation
UAT-13.2	MCP tool `search_codebase` exposed	Tool listed with correct JSON Schema parameters
UAT-13.3	MCP tool `read_file_context` executed	Returns file contents for requested path and line range
UAT-13.4	Model tool-call triggers local execution	Extension executes tool via VS Code API and returns result
UAT-13.5	Invalid tool call handled gracefully	Error returned to model without crashing extension

---

Test Group 14: Enterprise Graduation

Test ID	Scenario	Expected Result	Priority
UAT-14.1	SSO login via `vscode.authentication`	Extension obtains bearer token from session; attaches to requests	Must
UAT-14.2	`X-Request-Id` header present on requests	All outgoing calls include a UUID v4 `X-Request-Id` header	Must
UAT-14.3	Proxy settings respected	Requests routed through VS Code /	Should

Test ID	Scenario	Expected Result	Priority
		env var proxy configuration	
UAT-14.4	Switch from BYOK to SSO (zero code change)	Only `BASE_URL` and auth settings change; no extension code modified	Must

---

Test Group 15: Request Cancellation & Progress UX

Test ID	Scenario	Expected Result	Priority
UAT-15.1	Cancel streaming request via Stop button	Request aborted; UI cleared; “Request cancelled” message shown	Must
UAT-15.2	Cancel via Escape key	Same behavior as Stop button	Should
UAT-15.3	Progress indicator during long request	Elapsed time shown; cancellable indicator visible	Should
UAT-15.4	Cancellation cleans up HTTP connection	No lingering connections after cancellation	Should

---

Test Group 16: Concurrent Requests & Multi-Provider

Test ID	Scenario	Expected Result	Priority
UAT-16.1	Open two chat sessions simultaneously	Both sessions work independently without blocking	Must
UAT-16.2			Should

Test ID	Scenario	Expected Result	Priority
	Rapid-fire multiple requests	Requests queued; rate limits respected	
UAT-16.3	Configure multiple provider profiles	Profiles saved in settings correctly	Should
UAT-16.4	Switch active provider profile	New profile applied; subsequent requests use new provider	Should

---

Test Group 17: Context Window Overflow & Provider Capabilities

Test ID	Scenario	Expected Result	Priority
UAT-17.1	Prompt exceeds 90% of context window	Warning shown before sending request	Should
UAT-17.2	Provider returns `context_length_exceeded`	Automatic truncation attempted; retry succeeds	Should
UAT-17.3	Truncation notification shown to user	User informed of context reduction; log shows removed messages	Should
UAT-17.4	Provider lacks vision support	Feature disabled gracefully; user notified	Should
UAT-17.5	Provider capability detection cached	`/v1/models` called once per session, not per request	Should

---

Test Group 18: Extension Lifecycle & Cleanup

Test ID	Scenario	Expected Result	Priority
UAT-18.1	Reload extension during active request	Request cancelled; connections closed cleanly	Must
UAT-18.2	Deactivate extension	All event listeners disposed; no memory leaks	Must
UAT-18.3	IDE shutdown with active request	Request cancelled within 5s timeout	Should
UAT-18.4	Configuration schema migration (upgrade)	Old settings automatically migrated to new schema	Should

---

Test Group 19: Provider-Specific Compatibility

Test ID	Scenario	Expected Result	Priority
UAT-19.1	Configure Ollama provider (type: ollama, authMode: none)	Connection succeeds; no auth header sent	Must
UAT-19.2	Ollama streaming response	Tokens stream correctly; UI updates incrementally	Must
UAT-19.3	Anthropic provider via OpenAI-compatible endpoint	Chat completions succeed; tool calling works	Must
UAT-19.4			Should

Test ID	Scenario	Expected Result	Priority
	OpenShift AI provider (KServe/vLLM backend)	Model list retrieved; completions succeed	
UAT-19.5	AWS Bedrock via OpenAI-compatible gateway	Requests routed correctly; responses parsed	Should

---

Test Group 20: Security Edge Cases

Test ID	Scenario	Expected Result	Priority
UAT-20.1	Prompt injection via malicious HTML response	Response content-type validation fails; request rejected (REQ-API-006)	Must
UAT-20.2	SSO token expires during streaming request	Request fails gracefully; user prompted to re-authenticate	Must
UAT-20.3	SSO token refresh during active session	New token obtained transparently; request continues	Should
UAT-20.4	Custom CA certificate configured (corporate proxy)	HTTPS connection succeeds with corporate CA (REQ-CRED-003)	Must
UAT-20.5	CA certificate file missing or invalid	Clear error shown with CA setup instructions (REQ-CRED-005)	Should
UAT-20.6	API key deleted via	Credential removed from SecretStorage; subsequent	Must

Test ID	Scenario	Expected Result	Priority
	“Delete API Key” command	requests fail (REQ-CRED-012a)	
UAT-20.7	SecretStorage unavailable, OS stores unavailable	Persistent warning shown; env var fallback documented (REQ-CRED-007a)	Must
UAT-20.8	X-Request-Id header uniqueness	1000 concurrent requests all have unique UUIDs (REQ-ENT-003)	Should

---

Test Group 21: Tool Execution Safety

Test ID	Scenario	Expected Result	Priority
UAT-21.1	Tool-call loop exceeds maxLoopIterations (default 25)	Loop terminated; error returned to model (REQ-TOOL-007)	Must
UAT-21.2	Individual tool execution exceeds timeout (default 30s)	Tool cancelled; timeout error returned (REQ-TOOL-008)	Must
UAT-21.3	Destructive tool requires approval (REQ-TOOL-009)	User prompt shown before execution; requires confirmation	Must
UAT-21.4	Destructive tool in untrusted workspace	Tool execution blocked; read-only context only (REQ-ENT-004b)	Must
UAT-21.5	Tool result exceeds 10,000 characters	Result truncated with notice before sending to model (REQ-PROMPT-007)	Must

---

Test Group 22: Performance & NFR Validation

Test ID	Scenario	Expected Result	Priority
UAT-22.1	Measure extension activation time	< 2 seconds from activation trigger to ready state	Must
UAT-22.2	Measure extension memory footprint (idle)	< 100MB in IDE memory profiler	Must
UAT-22.3	Measure CPU usage (idle with no requests)	< 5% average CPU usage over 5 minutes	Should
UAT-22.4	Measure CPU usage (active streaming)	< 20% average during 60s streaming response	Should
UAT-22.5	Measure time to first token (after provider responds)	< 500ms from SSE first byte to UI display	Must
UAT-22.6	Measure extension overhead (request latency)	< 50ms overhead excluding network/provider time	Should
UAT-22.7	Extension bundle size (VS Code)	< 10MB .vsix package size	Should
UAT-22.8	Extension bundle size (Eclipse)	< 15MB with dependencies	Should
UAT-22.9	Network timeout: connection establishment	Fails after 10 seconds (PERF-006)	Must

Test ID	Scenario	Expected Result	Priority
UAT-22.10	Network timeout: streaming read	Fails after 60 seconds inactivity (PERF-006)	Must
UAT-22.11	Network timeout: non-streaming read	Fails after 30 seconds (PERF-006)	Must
UAT-22.12	Response size limit enforcement	1MB response truncated with warning (PERF-007)	Must

---

Test Group 23: Accessibility Compliance

Test ID	Scenario	Expected Result	Priority
UAT-23.1	Keyboard-only navigation (Command Palette)	All AI commands accessible via keyboard (A11Y-001)	Must
UAT-23.2	Screen reader announces streaming status	“Thinking…”, “Generating response…”, “Response complete” announced (A11Y-002)	Must
UAT-23.3	Focus management during streaming	Focus predictable; status changes announced	Must
UAT-23.4	High contrast theme support	UI elements visible in high contrast mode	Should
UAT-23.5	Zoom level 200% (VS Code)	UI remains usable at 200% zoom	Should

---

Test Group 24: Telemetry & Observability

Test ID	Scenario	Expected Result	Priority
UAT-24.1	Telemetry disabled by default	No network calls to telemetry endpoints without opt-in (SEC-004)	Must
UAT-24.2	Telemetry opt-in via settings UI	Setting enabled; events collected (TELEM-002)	Should
UAT-24.3	Telemetry event schema validation	Events match defined schema: ai.request.started, completed, failed, tool.executed (TELEM-001)	Should
UAT-24.4	Telemetry excludes PII	No prompts, API keys, file paths in telemetry payloads (TELEM-001)	Must
UAT-24.5	X-Request-Id in all LLM requests	UUID v4 header present in captured network traffic (REQ-ENT-003)	Must

---

Test Group 25: Internationalization Readiness

Test ID	Scenario	Expected Result	Priority
UAT-25.1	English (en-US) locale default	All UI strings display correctly in English (I18N-002)	Must
UAT-25.2	Code inspection: externalized strings	All user-facing strings use i18n framework (I18N-001)	Should
UAT-25.3	Message key convention validation	Keys follow `extension.category.message` pattern (I18N-002)	Should

---

Test Group 26: Architectural Hedge Validation

Test ID	Scenario	Expected Result	Priority
UAT-26.1	ToolOrchestrator interface abstraction	Tool-call loop implementation behind interface/abstract class (REQ-ARCH-001)	Must
UAT-26.2	thread_id assignment on new conversations	Every conversation assigned UUID v4 thread_id (REQ-ARCH-002)	Must
UAT-26.3	Event-typed streaming layer	Streaming emits typed events (token, tool_call, tool_result, status, error) not raw strings (REQ-ARCH-003)	Must
UAT-26.4	Multi-step UI progress display	UI shows step-by-step tool-call progress, not single progress bar (REQ-ARCH-004)	Should

---

Test Coverage Summary

Requirements Coverage

Requirement Category	Total Requirements	Covered by UAT	Coverage %
Configuration (CFG)	6	6	100%
Credentials (CRED)	11	11	100%
API (API)	8	8	100%
Provider (PROV)	11	11	100%
UX (UX)	10	10	100%
Prompt Engineering (PROMPT)	8	8	100%

Requirement Category	Total Requirements	Covered by UAT	Coverage %
JSON/Structured Output (JSON)	4	4	100%
Tool Calling (TOOL)	10	10	100%
MCP (MCP)	6	6	100%
Enterprise (ENT)	6	6	100%
Lifecycle (LIFE)	3	3	100%
Architecture (ARCH)	16	16	100%
Total Functional	**99**	**99**	**100%**
Non-Functional (NFRs)	**26**	**26**	**100%**

Test Execution Plan

Phase 1: Core Functionality (UAT Groups 1-10) - Focus: Installation, configuration, basic AI features, error handling - Timeline: Week 1-2 of QA - Pass criteria: 100% of “Must” priority tests pass

Phase 2: Advanced Features (UAT Groups 11-19) - Focus: Provider compatibility, MCP, enterprise features, tool calling - Timeline: Week 3-4 of QA - Pass criteria: 95% of “Must” tests, 80% of “Should” tests pass

Phase 3: Quality & Compliance (UAT Groups 20-26) - Focus: Security, performance, accessibility, i18n, architectural hedges - Timeline: Week 5-6 of QA - Pass criteria: 100% of security tests, 90% of NFR tests pass

---

← Previous | Back to main | Next →

Prompt Engineering & API Patterns

This section defines how developers building features within our extensions should construct and manage AI interactions. These patterns ensure consistency, reusability, and reliable outputs.

---

1. Message Roles

The OpenAI Chat Completions API uses distinct message roles. Our extension SDK must properly support all roles:

Role	Purpose	When to Use
system	Sets behavior, persona, and constraints for the model	Once at the start of each conversation/request; defines “who” the AI is
user	Contains the actual user request or input	For prompts derived from user actions or code context
assistant	Contains previous AI responses	For multi-turn conversations; providing examples (few-shot)
tool	Contains tool execution results	Sending function call results back to the model

Example: Programmatic Prompt Construction

```
// Code review feature - programmatic prompt (not interactive
chat)
const messages: ChatMessage[] = [
  {
    role: "system",
    content: `You are a senior software engineer performing
code review.
Focus on: security vulnerabilities, performance issues, and
maintainability.
Be concise. Use bullet points. Do not rewrite the code unless
asked.`
  },
  {
    role: "user",
    content: `Review this code:\n\n\`\`\`${language}\n$
{codeSnippet}\n\`\`\``
  }
];
```

---

2. Prompt Templates

For reusable, programmatic prompts (not user-authored chat), the extension should support templated prompts:

Template Structure

```
# Prompt template: code-review.yaml
id: code-review
name: "Code Review"
description: "Analyze code for issues and improvements"
version: "1.0"

system_prompt: |
  You are a senior software engineer performing code review.
  Focus on: {{focus_areas}}
  Language: {{language}}
  Be concise. Use bullet points.

user_prompt_template: |
  Review this {{language}} code:

  ```{{language}}
  {{code}}
```

{{#if specific_concern}} Pay special attention to: {{specific_concern}} {{/if}}

parameters: - name: language type: string required: true - name: code type: string required: true - name: focus_areas type: string default: “security, performance, maintainability” - name: specific_concern type: string required: false

model_settings: temperature: 0.3 # Lower for more consistent analysis max_tokens: 1024


```
**Programmatic Usage**

```typescript
const result = await ai.executeTemplate("code-review", {
  language: "typescript",
  code: selectedText,
  focus_areas: "security, error handling",
```

```
  specific_concern: "SQL injection"
});
```

---

3. Instruction Following Best Practices

To ensure reliable model outputs, prompts should follow these patterns:

Pattern	Description	Example
Role Definition	Clearly define who/what the model is	“You are a TypeScript expert…”
Task Specification	Explicitly state the task	“Your task is to refactor this function to…”
Output Format	Specify desired format	“Respond with a JSON object containing…”
Constraints	Define what NOT to do	“Do not include explanations. Only output code.”
Examples (Few-shot)	Provide input/output examples	Include 1-3 examples in system or user message

Anti-patterns to Avoid

Anti-pattern	Problem	Better Approach
Vague instructions	Inconsistent outputs	Be specific: “List exactly 3 issues”
No output format	Hard to parse results	Always specify format
Overly long prompts	Token waste, confusion	Keep focused; use separate calls for different tasks
Prompt injection risk	User input may override instructions	Clearly delimit user content with markers

---

4. Structured Output Handling

For features requiring parseable responses, use the `response_format` parameter or explicit JSON instructions:

Method 1: JSON Mode (Recommended)

```
const response = await ai.complete({
  messages: [...],
  response_format: { type: "json_object" },
  // When using JSON mode, instruct the model what JSON to
produce
});

// System prompt must mention JSON:
// "Respond with a JSON object containing: { issues: [...],
summary: string }"
```

Method 2: Explicit Schema in Prompt

```
const systemPrompt = `
Analyze the code and respond with this exact JSON structure:
{
  "issues": [
    {
      "severity": "high" | "medium" | "low",
      "line": number,
      "description": string,
      "suggestion": string
    }
  ],
  "overall_quality": "good" | "needs_work" | "poor",
  "summary": string
}

Respond ONLY with valid JSON. No additional text.
`;
```

Response Parsing

```
interface CodeReviewResult {
  issues: Array<{
    severity: "high" | "medium" | "low";
    line: number;
    description: string;
    suggestion: string;
  }>;
  overall_quality: "good" | "needs_work" | "poor";
  summary: string;
}

const result = await ai.complete<CodeReviewResult>({
  messages,
  response_format: { type: "json_object" },
  responseSchema: CodeReviewResultSchema  // Runtime
validation
});
```

---

5. Function Calling (Tools)

For features that need the model to invoke defined functions:

Defining Tools

```
const tools: Tool[] = [
  {
    type: "function",
    function: {
      name: "get_file_contents",
      description: "Read the contents of a file in the
workspace",
      parameters: {
        type: "object",
        properties: {
          file_path: {
            type: "string",
            description: "Relative path to the file"
          }
```

```
        },
        required: ["file_path"]
      }
    }
  },
  {
    type: "function",
    function: {
      name: "run_terminal_command",
      description: "Execute a terminal command. **WARNING:**
This tool executes arbitrary commands and is classified as
DESTRUCTIVE. Requires explicit user approval when
`requireApprovalForDestructiveTools` is enabled. Not exposed
to external MCP hosts by default.",
      parameters: {
        type: "object",
        properties: {
          command: { type: "string", description: "Shell
command to execute" },
          working_directory: { type: "string", description:
"Working directory (optional)" }
        },
        required: ["command"]
      }
    }
  }
];
```

Handling Tool Calls

```
const response = await ai.complete({
  messages,
  tools,
  tool_choice: "auto"  // or "required" or { type:
"function", function: { name: "..." } }
});

if (response.tool_calls) {
  for (const toolCall of response.tool_calls) {
    const args = JSON.parse(toolCall.function.arguments);
```

```
    // Execute the tool
    const result = await
executeFunction(toolCall.function.name, args);

    // Send result back to model
    messages.push(
      { role: "assistant", content: null, tool_calls:
[toolCall] },
      { role: "tool", tool_call_id: toolCall.id, content:
JSON.stringify(result) }
    );
  }

  // Continue conversation with tool results
  const finalResponse = await ai.complete({ messages,
tools });
}
```

Tool Execution Loop

For multi-turn tool calling until the model produces a final text response:

```
async function executeWithTools(
  initialMessages: ChatMessage[],
  tools: Tool[],
  maxIterations: number = 25
): Promise<string> {
  let messages = [...initialMessages];
  let iterations = 0;

  while (iterations < maxIterations) {
    const response = await ai.complete({ messages, tools,
tool_choice: "auto" });

    if (!response.tool_calls || response.tool_calls.length
=== 0) {
      // Final text response received
      return response.content;
    }

    // Execute tool calls
    messages.push({ role: "assistant", content: null,
```

```
tool_calls: response.tool_calls });

    for (const toolCall of response.tool_calls) {
      const result = await executeTool(toolCall);
      messages.push({
        role: "tool",
        tool_call_id: toolCall.id,
        content: JSON.stringify(result)
      });
    }

    iterations++;
  }

  throw new Error(`Tool execution loop exceeded maximum ${maxIterations} iterations`);
}
```

---

6. Context Management

For features that incorporate workspace context (code, files, symbols):

Context Type	How to Include	Token Considerations
Selected Code	Include in user message with language markers	Count tokens; truncate if needed
File Contents	Include relevant portions, not entire files	Summarize or chunk large files
Symbol Definitions	Include type signatures, interfaces	Prioritize relevant symbols
Error Messages	Include full error with stack trace	May truncate very long traces
Documentation	Include relevant doc snippets	Use RAG pattern for large docs

Context Prioritization

```
interface ContextBuilder {
  // Add context with priority (higher = more important to
keep)
  addContext(content: string, priority: number, label:
string): void;

  // Build final context respecting token budget
  build(maxTokens: number): string;
}

const context = new ContextBuilder();
context.addContext(errorMessage, 100, "error");      //
Highest priority
context.addContext(selectedCode, 90, "selection");    // High
context.addContext(fileContents, 50, "file");         //
Medium
context.addContext(relatedTypes, 30, "types");        //
Lower

const finalContext =
context.build(4000);  // Fits in context window
```

Context Window Management

Token Estimation:

```
// Rough estimation: 1 token ≈ 4 characters or 0.75 words in
English
function estimateTokens(text: string): number {
  return Math.ceil(text.length / 4);
}

// Use tiktoken library for accurate counting (model-
specific)
import { encoding_for_model } from "tiktoken";

function countTokens(text: string, model: string = "gpt-4"):
number {
  const enc = encoding_for_model(model);
  const tokens = enc.encode(text);
```

```
  enc.free();
  return tokens.length;
}
```

Truncation Strategies:

```
interface TruncationStrategy {
  // Remove oldest non-system messages
  truncateOldest(messages: ChatMessage[], targetTokens:
number): ChatMessage[];

  // Summarize long messages
  summarize(messages: ChatMessage[], targetTokens: number):
ChatMessage[];

  // Keep high-priority context, remove low-priority
  priorityBased(messages: ChatMessage[], priorities:
Map<string, number>, targetTokens: number): ChatMessage[];
}
```

---

7. Error Handling in Prompts

Handling Invalid Model Outputs

```
async function robustCompletion<T>(
  messages: ChatMessage[],
  schema: Schema,
  maxRetries: number = 2
): Promise<T> {
  for (let attempt = 0; attempt <= maxRetries; attempt++) {
    try {
      const response = await ai.complete({
        messages,
        response_format: { type: "json_object" }
      });

      // Validate against schema
      const parsed = JSON.parse(response.content);
      const validated = schema.validate(parsed);

      return validated as T;
```

```
    } catch (error) {
      if (attempt === maxRetries) {
        throw new Error(`Failed to get valid response after ${maxRetries + 1} attempts: ${error.message}`);
      }

      // Add correction instruction for next attempt
      messages.push(
        { role: "assistant", content: response.content },
        { role: "user", content: `The response was invalid: ${error.message}. Please correct and respond with valid JSON.` }
      );
    }
  }
}
```

Rate Limit Handling

```
async function completionWithBackoff(
  messages: ChatMessage[],
  maxRetries: number = 3
): Promise<ChatResponse> {
  let lastError: Error;

  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      return await ai.complete({ messages });
    } catch (error) {
      if (error.status === 429) {
        // Rate limited - exponential backoff
        const delayMs = Math.pow(2, attempt) * 1000;
        await sleep(delayMs);
        lastError = error;
      } else {
        throw error;
      }
    }
  }
```

```
  throw lastError;
}
```

---

8. Best Practices Summary

DO's

✅ **Clear Instructions**: Always provide explicit, unambiguous instructions
✅ **Output Format**: Specify the exact format you expect (Markdown, JSON, code block)
✅ **Delimit User Content**: Use clear markers (e.g., triple backticks, XML tags) to separate user input
✅ **Few-shot Examples**: Include 1-3 examples when format matters
✅ **Token Awareness**: Estimate tokens before sending; truncate intelligently
✅ **Error Handling**: Validate outputs; implement retry logic with backoff
✅ **Context Prioritization**: Include most important context first
✅ **Streaming**: Use SSE streaming for better UX on long responses
✅ **Tool Descriptions**: Write clear, detailed descriptions for function calling

DON'Ts

❌ **Vague Prompts**: Avoid "analyze this code" without specifics
❌ **No Validation**: Never trust model output without validation
❌ **Ignore Token Limits**: Don't send prompts that exceed context window
❌ **Hardcode Model Names**: Use configuration for model selection
❌ **Log Sensitive Data**: Never log API keys, user code, or completions
❌ **Block UI**: Always use async/streaming to avoid freezing the IDE
❌ **Assume Perfect Output**: Models can hallucinate or produce invalid formats
❌ **Prompt Injection**: Never directly concatenate untrusted user input into system prompts

---

9. Example Patterns

Code Generation

```
const messages = [
  {
    role: "system",
```

```
    content: "You are a TypeScript expert. Generate
production-quality code with error handling and TypeScript
types."
  },
  {
    role: "user",
    content: `Generate a function that ${userRequest}.
\n\nRequirements:\n- Use TypeScript with strict types\n-
Include error handling\n- Add JSDoc comments\n\nRespond with
ONLY the code, no explanations.`
  }
];
```

Code Explanation

```
const messages = [
  {
    role: "system",
    content: "You are a senior software engineer explaining
code to junior developers. Be clear and concise."
  },
  {
    role: "user",
    content: `Explain what this code does:\n\n\`\`\`${
{language}\n${code}\n\`\`\`\n\nUse bullet points. Focus on
the main logic.`
  }
];
```

Bug Diagnosis

```
const messages = [
  {
    role: "system",
    content: "You are a debugging expert. Analyze errors and
suggest fixes."
  },
  {
    role: "user",
    content: `I'm getting this error:\n\n\`\`\`\n${
{errorMessage}\n\`\`\`\n\nIn this code:\n\n\`\`\`${language}
\n${code}
```

```
\n\`\`\`\n\nWhat's the likely cause and how do I fix it?`
  }
];
```

Refactoring Suggestions

```
const messages = [
  {
    role: "system",
    content: "You are a code quality expert. Suggest
improvements for maintainability, performance, and best
practices."
  },
  {
    role: "user",
    content: `Suggest improvements for this code:\n\n\`\`\`$
{language}\n${code}\n\`\`\`\n\nFocus on: $
{focusAreas.join(", ")}`
  }
];
```

← Previous | Back to main | Next →

Document Control

This section contains version history, approvals, and licensing information for this Software Requirements Specification.

Document Information

Document ID: SRS-AI-CLIENT-001
Version: 0.17
Date: 2026-02-26
Standard: ISO/IEC/IEEE 29148:2018
Status: Draft
Author: Mark Sigler

Revision History

Version	Date	Author	Changes
0.1	2026-02-01	Mark Sigler	Initial document structure and executive summary
0.2	2026-02-02	Mark Sigler	Added personas, goals, and assumptions
0.3	2026-02-03	Mark Sigler	Added data flow architecture and security considerations
0.4	2026-02-04	Mark Sigler	Added functional requirements (initial set)
0.5	2026-02-05	Mark Sigler	Added configuration schema and error handling
0.6	2026-02-06	Mark Sigler	Added UAT plan
0.7	2026-02-07	Mark Sigler	Added MCP support, BYOK via SecretStorage, Enterprise SSO graduation path
0.8	2026-02-08	Mark Sigler	Added prompt engineering patterns and structured output requirements
0.9	2026-02-09	Mark Sigler	Added function calling (tools) requirements and examples
0.10	2026-02-10	Mark Sigler	Added system architecture

Version	Date	Author	Changes
			diagrams and component overview
0.11	2026-02-11	Mark Sigler	Added image generation requirements (limited scope: single image generation only)
0.12	2026-02-12	Mark Sigler	Added design-phase requirements (§7) — provider-agnostic interface, BYOK, MCP, enterprise graduation, architectural hedges
0.13	2026-02-13	Mark Sigler	P4 Priority tier: added 16 new requirements (REQ-68 through REQ-78, REQ-79-85) for URL path construction, tool execution safety, user interaction patterns, concurrent requests, provider capability detection, multi-provider config, context window overflow handling
0.14	2026-02-14	Mark Sigler	P4 follow-up: added 3 extension lifecycle requirements (REQ-89, REQ-90, REQ-99)
0.15	2026-02-15	Mark Sigler	P4 NFR additions: added 9 new NFR rows and

Version	Date	Author	Changes
			requirements (bundle size limit, CPU usage limit, i18n support, accessibility compliance, telemetry event schema, telemetry settings UI, SecretStorage API unavailable degradation path, response size limit, network timeouts, activation event constraints) — added REQ-07a, REQ-91-99
0.16	2026-02-16	Mark Sigler	P5 Test Coverage: added 8 new UAT test groups (19-26) covering provider-specific compatibility, security edge cases, tool safety, performance/NFR validation, accessibility, telemetry, i18n, architectural hedge validation (47 new test scenarios)
0.17	2026-02-26	Mark Sigler	**IEEE 29148 Migration**: Restructured document into ISO/IEC/IEEE 29148:2018 format with modular file organization. All 99

Version	Date	Author	Changes
			requirements remapped to domain-prefixed IDs. Added formal traceability matrix. No content removed.

Document Approvals

Status: Draft — pending final stakeholder review

Role	Name	Date	Signature
Product Owner	Mark Sigler	2026-02-26	[Approved]
Development Lead	[Pending]	[Pending]	[Pending]
QA Lead	[Pending]	[Pending]	[Pending]
Technical Architect	[Pending]	[Pending]	[Pending]

Change Control Process

For Document Versions 0.x (Draft): - Changes may be made without formal approval - Version number incremented with each substantive change - Change log maintained in Revision History table

For Document Versions 1.x+ (Released): - All changes require approval from Product Owner - Breaking changes (major version) require approval from all stakeholders - Non-breaking changes (minor version) require approval from Product Owner and Development Lead - Change requests must be documented with rationale and impact assessment

---

License and Attribution

License

This document is licensed under the **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)** license.

You are free to: - **Share** — copy and redistribute the material in any medium or format - **Adapt** — remix, transform, and build upon the material for any purpose, even commercially

Under the following terms: - **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made - **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original

TASL Attribution

Title: AI Client Software Requirements Specification
Author: Mark Sigler
Source: ai-client-specs *(update with actual repository URL)*
License: CC BY-SA 4.0

Full License Text: See LICENSE.md in the project root directory.

Third-Party Components and Dependencies

This SRS references several third-party technologies and standards:

Standards and Specifications

Component	Source	License/Terms
ISO/IEC/IEEE 29148:2018	International Organization for Standardization	ISO Copyright
OpenAI Chat Completions API	OpenAI	OpenAI Terms
Model Context Protocol (MCP)	Anthropic	Apache 2.0

Component	Source	License/Terms
EARS (Easy Approach to Requirements Syntax)	Alistair Mavin et al.	Public domain methodology

Software Libraries (Referenced in Requirements)

Component	Purpose	License
VS Code Extension API	IDE integration	MIT License
Eclipse RCP/JFace	IDE integration	Eclipse Public License 2.0
SecretStorage API (VS Code)	Credential management	MIT License
Eclipse Secure Storage	Credential management	Eclipse Public License 2.0

Note: This SRS does not distribute these components. They are referenced as integration dependencies only. Implementers must comply with each component's respective license terms.

Compliance and Standards

This document is structured in accordance with:

- **ISO/IEC/IEEE 29148:2018** — Systems and software engineering — Life cycle processes — Requirements engineering
- **EARS (Easy Approach to Requirements Syntax)** — Requirements writing methodology
- **MoSCoW Prioritization** — Must have, Should have, Could have, Won't have

Requirements Traceability

All requirements defined in this specification are: - Uniquely identified with domain-prefixed IDs (e.g., REQ-CFG-001, REQ-API-005) - Written using EARS syntax patterns (Ubiquitous, Event-driven, Unwanted behavior, State-driven, Optional) -

Mapped to User Acceptance Test cases in Appendix A: Traceability Matrix - Prioritized using MoSCoW framework

Test Coverage

All functional requirements (99 total) and all non-functional requirements (26 total) are covered by User Acceptance Test cases defined in Verification.

Coverage Statistics: - Functional Requirements Coverage: **100%** (99/99) - Non-Functional Requirements Coverage: **100%** (26/26) - Total Test Cases: **130+ scenarios** across 26 test groups

---

Related Documents

- **Introduction** — Purpose, scope, and context
- **Overall Description** — Product perspective and constraints
- **System Features** — Complete functional requirements (99 requirements)
- **External Interfaces** — Interface specifications
- **Non-Functional Requirements** — Quality attributes (26 NFRs)
- **Other Requirements** — Database, i18n, legal
- **Appendix A: Traceability Matrix** — Requirements ? Tests mapping
- **Appendix B: Glossary** — Definitions and terminology
- **Appendix C: Analysis Models** — Architecture diagrams
- **API Patterns** — Developer implementation guide
- **Verification** — User Acceptance Testing plan

Contact Information

Document Owner: Mark Sigler
Email: Contact via LinkedIn
Organization: [Your Organization Name]
Project Repository: [GitHub/GitLab repository URL]

For questions, clarifications, or change requests, please contact the document owner or submit an issue in the project repository.

← Previous | Back to main